

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

*



BÁO CÁO MÔN HỌC IT3160
WORD-LEVEL HANDWRITTEN TEXT RECOGNITION
NHẬN DẠNG CHỮ VIẾT TAY MỨC ĐỘ TỪ ĐƠN

Giảng viên phụ trách: PGS. TS. Ngô Văn Linh

Mã lớp: IT3160

Danh sách sinh viên:

STT	Họ tên	Mã sinh viên	Lớp
1	Nguyễn Trí Hiếu	202416203	CTTN KHMT K69
2	Đỗ Hải Đăng	202400035	CTTN KHMT K69
3	Đinh Thái Sơn	202416746	CTTN KHMT K69

Hà Nội, tháng 6 năm 2026

TÓM TẮT BÁO CÁO

Báo cáo trình bày quá trình xây dựng hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ sử dụng PyTorch, với đầu vào là ảnh chứa một từ tiếng Anh viết tay và đầu ra là chuỗi ký tự tương ứng. Dự án được thực hiện trên IAM Handwriting Dataset ở mức word-level, gồm 96,835 mẫu sau khi làm sạch.

Trong phạm vi bài tập lớn, nhóm xây dựng và so sánh hai kiến trúc: CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss; Attention Model sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder. Trên tập kiểm thử IAM, CTC Baseline đạt 80.75% Word Accuracy và 7.27% Character Error Rate, trong khi Attention Model đạt 83.09% Word Accuracy và 7.06% Character Error Rate. Kết quả cho thấy Attention Model có cải thiện so với baseline trong cùng điều kiện đánh giá, nhưng khả năng tổng quát hóa trên ảnh chữ viết tay thực tế vẫn còn là một hạn chế.

Mục lục

Tóm tắt báo cáo	i
1 Giới thiệu đề tài	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Mục tiêu và phạm vi thực hiện	1
1.3 Định hướng tiếp cận	2
1.4 Ý nghĩa và cấu trúc báo cáo	2
2 Cơ sở lý thuyết	3
2.1 Bài toán nhận dạng chữ viết tay	3
2.2 Trích xuất đặc trưng ảnh và mô hình hóa chuỗi	4
2.3 Mô hình CTC	4
2.4 Mô hình Encoder–Decoder với Attention	5
2.5 Các chỉ số đánh giá	7
2.6 Tổng kết chương	7
3 Dữ liệu và tiền xử lý	8
3.1 Bộ dữ liệu IAM word-level	8
3.2 Làm sạch nhãn, xây dựng từ vựng và chia tập dữ liệu	9
3.3 Tiền xử lý và tăng cường dữ liệu	10
3.4 Dataset, DataLoader và pipeline chuẩn bị dữ liệu	12
3.5 Hạn chế dữ liệu và tổng kết chương	12
4 Kiến trúc mô hình	13
4.1 Tổng quan hệ thống và xử lý đặc trưng	13
4.2 Mô hình CTC Baseline	15
4.3 Mô hình Attention	17
4.4 So sánh hai mô hình và tổ chức suy diễn	18
4.5 Tổng kết chương	19
5 Cài đặt hệ thống	20
5.1 Môi trường phát triển	20
5.2 Phần cứng sử dụng	20
5.3 Cấu trúc thư mục dự án	21
5.4 Vai trò của các thư mục chính	21
5.5 Pipeline chuẩn bị dữ liệu	23
5.6 Pipeline huấn luyện	24
5.7 Chiến lược lưu checkpoint	25
5.8 Pipeline đánh giá	25
5.9 Pipeline dự đoán ảnh thực tế	26
5.10 Đầu ra của hệ thống	27

5.11	Quản lý phụ thuộc	27
5.12	Luồng chạy hệ thống	28
5.13	Tính mô-đun của hệ thống	28
5.14	Một số lưu ý khi triển khai	29
5.15	Tổng kết chương	29
6	Thực nghiệm và kết quả	30
6.1	Thiết lập thực nghiệm	30
6.2	Kết quả trên tập kiểm thử IAM	31
6.3	Phân tích kết quả	31
6.4	Kiểm thử thực tế và phân tích lỗi	33
6.5	Tổng kết chương	33
7	Kết luận và hướng phát triển	34
7.1	Kết luận	34
7.2	Những kết quả đạt được	34
7.3	Hạn chế của dự án	35
7.4	Bài học rút ra	35
7.5	Hướng phát triển	36
7.6	Kết luận chung	37
	Tài liệu tham khảo	38

Danh sách hình vẽ

2.1	Các mức độ của bài toán nhận dạng chữ viết tay	3
2.2	Minh họa ý tưởng căn chỉnh trong CTC	5
2.3	Minh họa cơ chế attention trong mô hình Encoder–Decoder	6
3.1	Một số ảnh mẫu từ IAM Handwriting Dataset ở mức word-level	8
3.2	Tỉ lệ phân chia dữ liệu train, validation và test	10
3.3	Pipeline tiền xử lý ảnh đầu vào	10
3.4	Ví dụ minh họa một số phép tăng cường dữ liệu	11
4.1	Pipeline tổng quan của hệ thống nhận dạng chữ viết tay	13
4.2	Minh họa quá trình chuyển feature map 2D thành chuỗi đặc trưng	15
4.3	Kiến trúc mô hình CTC Baseline	16
4.4	Kiến trúc mô hình Attention	17
5.1	Cấu trúc thư mục tổng quát của dự án	21
5.2	Luồng chuẩn bị dữ liệu trước khi huấn luyện	23
5.3	Pipeline suy diễn trên ảnh thực tế	27
5.4	Luồng chạy tổng quát của hệ thống	28
6.1	Biểu đồ so sánh kết quả CTC Baseline và Attention Model	31
6.2	Ví dụ attention map mô hình Attention	32

Danh sách bảng

3.1	Phân chia bộ dữ liệu IAM sau tiền xử lý	9
3.2	Kích thước ảnh đầu vào của hai mô hình	11
4.1	So sánh kiến trúc CTC Baseline và Attention Model	19
5.1	Môi trường phát triển của dự án	20
5.2	Phần cứng sử dụng trong quá trình huấn luyện	20
6.1	Phân chia dữ liệu trong thực nghiệm	30
6.2	Tóm tắt thiết lập thực nghiệm	30
6.3	Kết quả so sánh hai mô hình trên IAM Test Set	31
6.4	Kết quả chi tiết của hai mô hình	31
6.5	Một số ví dụ dự đoán của hai mô hình	32

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Trong quá trình số hóa tài liệu, nhu cầu chuyển đổi nội dung từ ảnh sang văn bản có thể chỉnh sửa được ngày càng phổ biến. Các hệ thống OCR hiện nay đã xử lý khá tốt nhiều loại văn bản in, biểu mẫu và tài liệu lưu trữ. Tuy nhiên, với chữ viết tay, bài toán vẫn khó hơn đáng kể do nét chữ thay đổi theo từng người viết, từng điều kiện ghi/chụp và cả chất lượng ảnh đầu vào.

Chữ viết tay không có một khuôn dạng cố định. Cùng một ký tự có thể được viết nghiêng hơn, cao hơn, dính nét hơn hoặc mảnh hơn tùy theo thói quen của người viết.

Nhóm lựa chọn đề tài *Word-Level Handwritten Text Recognition* vì bài toán này kết hợp nhiều nội dung quan trọng trong học sâu, từ xử lý ảnh, trích xuất đặc trưng cho đến mô hình hóa và giải mã chuỗi ký tự. Khác với phân loại ảnh thông thường, hệ thống không chỉ chọn một nhãn cố định mà phải sinh ra một chuỗi ký tự có độ dài thay đổi.

Trong phạm vi bài tập lớn, nhóm sử dụng IAM Handwriting Dataset ở mức word-level để huấn luyện và đánh giá mô hình. Trên cùng bộ dữ liệu này, nhóm so sánh hai hướng tiếp cận: mô hình CTC và mô hình Encoder-Decoder có Attention.

1.2. Mục tiêu và phạm vi thực hiện

Mục tiêu chính của dự án là xây dựng một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ chạy local bằng PyTorch. Hệ thống nhận đầu vào là ảnh chứa một từ tiếng Anh viết tay, sau đó tiền xử lý ảnh, đưa qua mô hình học sâu và trả về chuỗi ký tự dự đoán.

Các mục tiêu cụ thể gồm:

- Tìm hiểu bài toán nhận dạng chữ viết tay mức độ từ đơn lẻ.
- Chuẩn bị và tiền xử lý IAM Handwriting Dataset ở mức word-level.
- Xây dựng pipeline dữ liệu phục vụ huấn luyện, kiểm định và kiểm thử.
- Cài đặt mô hình CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss.
- Cài đặt mô hình Attention sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.
- Huấn luyện, đánh giá và so sánh hai mô hình trên cùng tập dữ liệu.
- Đánh giá kết quả bằng Word Accuracy, Character Error Rate và Normalized Edit Distance.
- Hỗ trợ dự đoán ảnh đơn lẻ hoặc thư mục ảnh, đồng thời trực quan hóa attention map với mô hình Attention.

Dự án chỉ tập trung vào nhận dạng chữ viết tay ở mức độ từ đơn lẻ. Mỗi ảnh đầu vào được giả định chỉ chứa một từ tiếng Anh. Hệ thống không xử lý nhận dạng dòng văn bản, đoạn văn hoặc tài liệu hoàn chỉnh; các bước như phát hiện vùng chữ, tách dòng và tách từ không nằm trong phạm vi triển khai.

Toàn bộ quá trình huấn luyện và suy diễn được thực hiện trên môi trường local, không sử dụng các dịch vụ OCR API bên thứ ba. Các mô hình được xây dựng bằng PyTorch và đánh giá trên IAM Handwriting Dataset sau khi đã được làm sạch, chia thành các tập train, validation và test.

1.3. Định hướng tiếp cận

Pipeline tổng quát của hệ thống gồm các bước: tiền xử lý ảnh, trích xuất đặc trưng bằng CNN, mô hình hóa chuỗi, giải mã ký tự và đánh giá kết quả.

Ảnh đầu vào được chuẩn hóa kích thước rồi đưa qua ResNet18 để trích xuất đặc trưng. Feature map từ CNN sau đó được chuyển thành chuỗi đặc trưng theo chiều ngang của ảnh, tương ứng với thứ tự xuất hiện của các ký tự trong từ.

Với CTC Baseline, chuỗi đặc trưng được đưa qua BiLSTM và lớp tuyến tính để dự đoán phân phối ký tự tại từng bước thời gian. CTC Loss được dùng để học căn chỉnh ngầm giữa chuỗi đặc trưng và nhãn.

Với Attention Model, chuỗi đặc trưng được xử lý thêm bằng BiLSTM Context Encoder. Sau đó, Bahdanau Attention và GRU Decoder được sử dụng để sinh chuỗi ký tự từng bước. Hướng tiếp cận này cho phép mô hình học căn chỉnh mềm giữa vùng đặc trưng ảnh và ký tự đang được dự đoán.

1.4. Ý nghĩa và cấu trúc báo cáo

Dự án giúp nhóm hiểu rõ hơn cách xây dựng một hệ thống học sâu cho bài toán nhận dạng chuỗi từ dữ liệu ảnh. Nhóm không chỉ sử dụng mô hình có sẵn mà tự triển khai các thành phần chính từ xử lý dữ liệu, thiết kế kiến trúc, huấn luyện, đánh giá đến suy diễn.

Việc so sánh CTC Baseline và Attention Model giúp làm rõ sự khác biệt giữa hai hướng tiếp cận phổ biến trong nhận dạng văn bản. CTC đơn giản, ổn định và không cần decoder tự hồi quy; Attention linh hoạt hơn, có khả năng học căn chỉnh động và có thể trực quan hóa vùng ảnh mà mô hình tập trung khi dự đoán.

Bên cạnh kết quả trên tập kiểm thử IAM, dự án cũng cho thấy một vấn đề thực tế: mô hình có thể đạt kết quả tốt trên bộ dữ liệu chuẩn nhưng vẫn suy giảm khi áp dụng cho ảnh chữ viết tay ngoài môi trường chuẩn. Điều này giúp nhóm nhìn nhận thận trọng hơn về khả năng tổng quát hóa của mô hình.

Báo cáo gồm 7 chương. Chương 1 giới thiệu đề tài, mục tiêu, phạm vi và định hướng tiếp cận. Chương 2 trình bày cơ sở lý thuyết liên quan đến HTR, CNN, BiLSTM, CTC, Attention và các chỉ số đánh giá. Chương 3 mô tả bộ dữ liệu IAM và quá trình tiền xử lý. Chương 4 trình bày kiến trúc hai mô hình. Chương 5 mô tả cách cài đặt hệ thống và pipeline thực hiện. Chương 6 trình bày kết quả thực nghiệm và phân tích. Chương 7 tổng kết dự án, nêu hạn chế và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Bài toán nhận dạng chữ viết tay

Nhận dạng chữ viết tay, hay Handwritten Text Recognition (HTR), là bài toán chuyển đổi ảnh chứa chữ viết tay thành chuỗi văn bản tương ứng. Đây là bài toán nằm giữa thị giác máy tính và xử lý chuỗi trong học sâu, vì mô hình vừa phải hiểu nội dung ảnh, vừa phải sinh ra chuỗi ký tự đúng thứ tự.

So với chữ in, chữ viết tay khó nhận dạng hơn do nét chữ thay đổi rất nhiều giữa các người viết. Cùng một ký tự có thể có nhiều hình dạng khác nhau, phụ thuộc vào thói quen viết, độ nghiêng, tốc độ viết, loại bút và chất lượng ảnh đầu vào. Vì vậy, mô hình không chỉ cần nhận ra đặc trưng hình ảnh cục bộ, mà còn cần khai thác quan hệ tuần tự giữa các ký tự trong từ.

Tùy theo phạm vi đầu vào, HTR có thể được chia thành nhiều mức: nhận dạng ký tự, nhận dạng từ, nhận dạng dòng và nhận dạng cả đoạn hoặc tài liệu. Trong dự án này, nhóm tập trung vào mức *word-level*, tức mỗi ảnh đầu vào chỉ chứa một từ tiếng Anh viết tay.

HTR Task Levels (Mức độ bài toán nhận dạng chữ viết tay)



Hình 2.1: Các mức độ của bài toán nhận dạng chữ viết tay

Về mặt hình thức, với một ảnh đầu vào X chứa một từ viết tay, mục tiêu của mô hình là dự đoán chuỗi ký tự:

$$Y = (y_1, y_2, \dots, y_L)$$

Trong đó L là độ dài của từ, còn mỗi y_i là một ký tự thuộc bộ từ vựng đã định nghĩa. Điểm khó của bài toán

là chuỗi đầu ra có độ dài thay đổi, trong khi ảnh đầu vào sau tiền xử lý thường có kích thước cố định.

Một khó khăn khác là dữ liệu không cung cấp vị trí chính xác của từng ký tự trong ảnh. Ta chỉ biết ảnh tương ứng với từ nào, chứ không biết ký tự thứ nhất, thứ hai hay thứ ba nằm ở vùng nào. Vì vậy, mô hình cần một cơ chế để học căn chỉnh ngầm hoặc căn chỉnh mềm giữa đặc trưng ảnh và chuỗi ký tự đầu ra.

Trong dự án, nhóm triển khai hai hướng tiếp cận chính: hướng dựa trên CTC Loss và hướng Encoder–Decoder có Attention.

2.2. Trích xuất đặc trưng ảnh và mô hình hóa chuỗi

Convolutional Neural Network (CNN) là kiến trúc mạng nơ-ron thường dùng trong xử lý ảnh. CNN học đặc trưng thông qua các lớp tích chập: các tầng đầu thường học cạnh, nét, góc; các tầng sâu hơn học những đặc trưng trừu tượng hơn như hình dạng ký tự hoặc cấu trúc nét chữ.

Trong bài toán HTR, CNN được dùng để biến ảnh đầu vào thành feature map. Feature map này chứa thông tin đặc trưng của ảnh và sẽ được chuyển thành chuỗi đặc trưng để đưa vào các mô hình xử lý tuần tự.

Dự án sử dụng ResNet18 làm CNN Encoder. Thay vì dùng ResNet18 như một mô hình phân loại ảnh hoàn chỉnh, nhóm chỉ sử dụng phần trích xuất đặc trưng của mạng. Với ảnh đầu vào, encoder tạo ra feature map có dạng:

$$F \in \mathbb{R}^{B \times C \times H' \times W'}$$

Trong đó B là batch size, C là số kênh đặc trưng, H' là chiều cao và W' là chiều rộng của feature map. Vì chữ tiếng Anh thường được đọc từ trái sang phải, chiều rộng W' có thể được xem như trục thời gian. Feature map sau đó được reshape thành chuỗi đặc trưng để đưa vào BiLSTM hoặc decoder.

Sau bước trích xuất đặc trưng, mô hình cần xử lý chuỗi đặc trưng theo thứ tự không gian của ảnh. Recurrent Neural Network (RNN) là nhóm mô hình được thiết kế cho dữ liệu tuần tự, nhưng RNN truyền thống thường gặp khó khăn khi học các phụ thuộc dài hạn. LSTM cải thiện vấn đề này nhờ các cổng điều khiển, giúp mô hình quyết định thông tin nào cần giữ lại, thông tin nào cần quên và thông tin nào cần đưa ra tại mỗi bước.

BiLSTM là phiên bản hai chiều của LSTM. Mô hình xử lý chuỗi theo cả chiều trái sang phải và phải sang trái, sau đó kết hợp thông tin từ hai hướng. Điều này hữu ích trong nhận dạng chữ viết tay, vì một ký tự thường được nhận diện tốt hơn khi xét cả ngữ cảnh trước và sau nó.

Trong dự án, BiLSTM được sử dụng ở cả hai mô hình. Với CTC Baseline, BiLSTM xử lý chuỗi đặc trưng trước khi dự đoán phân phối ký tự tại từng timestep. Với Attention Model, BiLSTM đóng vai trò Context Encoder, giúp đặc trưng đưa vào attention giàu thông tin ngữ cảnh hơn.

2.3. Mô hình CTC

Connectionist Temporal Classification (CTC) là một hàm mất mát thường dùng cho các bài toán nhận dạng chuỗi khi không có thông tin căn chỉnh trực tiếp giữa đầu vào và đầu ra. Trong HTR, ta biết một ảnh tương ứng với một từ, nhưng không biết chính xác ký tự nào nằm ở vị trí nào trong ảnh.

CTC giải quyết vấn đề này bằng cách cho phép mô hình sinh ra một chuỗi trung gian dài hơn nhãn thật. Chuỗi này có thể chứa ký tự lặp và một ký hiệu đặc biệt gọi là blank token. Sau đó, chuỗi được rút gọn bằng hai bước: gộp các ký tự giống nhau đứng liền nhau và loại bỏ blank token.

Ví dụ, nếu – là blank token, chuỗi dự đoán thô:

-hh-e-ll-oo-

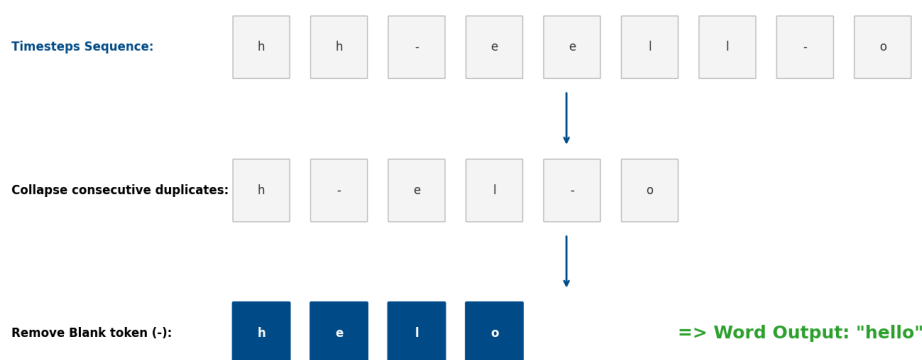
có thể được rút gọn thành:

helo

CTC Loss tính tổng xác suất của tất cả các chuỗi căn chỉnh hợp lệ có thể rút gọn về nhãn đúng. Nhờ đó, mô hình có thể được huấn luyện mà không cần gán nhãn vị trí từng ký tự trên ảnh.

Ưu điểm của CTC là đơn giản, ổn định và phù hợp với các bài toán có thứ tự đầu ra gần như đơn điệu theo đầu vào. Tuy nhiên, CTC cũng có hạn chế: mô hình khó khai thác trực tiếp quan hệ giữa các ký tự đã sinh trước đó, và độ dài đầu ra thường bị giới hạn bởi số timestep của feature map.

CTC Alignment & Decoding Mechanism (Cơ chế căn chỉnh CTC)



Hình 2.2: Minh họa ý tưởng căn chỉnh trong CTC

Trong quá trình suy diễn, mô hình CTC thường sử dụng CTC Greedy Decoding. Tại mỗi timestep, mô hình chọn ký tự có xác suất cao nhất, sau đó rút gọn chuỗi bằng cách loại bỏ ký tự lặp liên tiếp và blank token. Cách giải mã này nhanh và dễ triển khai, nhưng có thể bỏ qua những chuỗi có xác suất tổng thể tốt hơn.

2.4. Mô hình Encoder–Decoder với Attention

Attention là cơ chế cho phép mô hình tập trung vào các phần khác nhau của đầu vào khi sinh từng phần tử đầu ra. Thay vì nén toàn bộ ảnh vào một vector cố định, decoder có thể truy cập lại các đặc trưng encoder tại mỗi bước giải mã.

Trong bài toán nhận dạng chữ viết tay, khi dự đoán một ký tự, attention có thể học cách tập trung vào vùng ảnh tương ứng với ký tự đó. Đây là một dạng căn chỉnh mềm giữa chuỗi đặc trưng ảnh và chuỗi ký tự đầu ra.

Dự án sử dụng Bahdanau Attention. Tại bước giải mã t , mô hình tính điểm liên quan giữa trạng thái decoder trước đó s_{t-1} và từng vector encoder h_s :

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Các điểm này được chuẩn hóa bằng softmax để tạo trọng số attention:

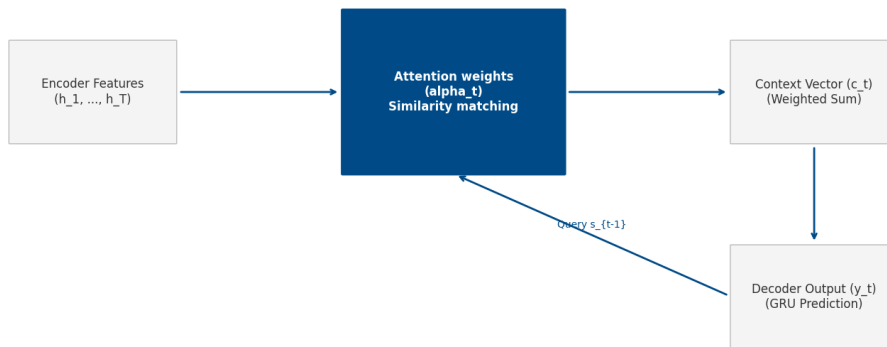
$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{k=1}^T \exp(e_{t,k})}$$

Sau đó, vector ngữ cảnh được tính bằng tổng có trọng số:

$$c_t = \sum_{s=1}^T \alpha_{t,s} h_s$$

Trong đó, $\alpha_{t,s}$ cho biết mức độ mô hình tập trung vào vị trí encoder s khi sinh ký tự ở bước t . Vector c_t được đưa vào decoder để dự đoán ký tự tiếp theo.

Bahdanau Attention Mechanism (Cơ chế chú ý Bahdanau)



Hình 2.3: Minh họa cơ chế attention trong mô hình Encoder–Decoder

Kiến trúc Encoder–Decoder là kiến trúc thường dùng trong các bài toán chuyển đổi chuỗi. Encoder mã hóa đầu vào thành biểu diễn đặc trưng, còn decoder sinh đầu ra từng bước dựa trên biểu diễn đó. Trong dự án này, encoder gồm CNN Encoder và BiLSTM Context Encoder. CNN trích xuất đặc trưng không gian từ ảnh, sau đó feature map được reshape thành chuỗi và đưa qua BiLSTM để bổ sung ngữ cảnh hai chiều.

Decoder sử dụng GRUCell để sinh chuỗi ký tự đầu ra. Tại mỗi bước, decoder nhận embedding của ký tự trước đó, vector ngữ cảnh từ attention và trạng thái ẩn hiện tại để dự đoán ký tự tiếp theo.

So với CTC, mô hình Encoder–Decoder với Attention linh hoạt hơn vì decoder có thể điều kiện hóa ký tự hiện tại dựa trên các ký tự đã sinh trước đó. Ngoài ra, attention map cũng có thể được trực quan hóa để quan sát vùng ảnh mà mô hình tập trung trong quá trình dự đoán.

Trong quá trình huấn luyện, Attention Model sử dụng Teacher Forcing. Thay vì luôn đưa ký tự mô hình dự đoán ở bước trước vào bước hiện tại, Teacher Forcing đôi khi sử dụng ký tự đúng từ ground truth. Cách làm này giúp mô hình học nhanh và ổn định hơn trong giai đoạn đầu, khi dự đoán của mô hình còn nhiều lỗi. Khi đánh giá và suy diễn, Teacher Forcing không được sử dụng vì mô hình phải tự sinh toàn bộ chuỗi dự

đoán.

Về giải mã, Attention Model có thể dùng Greedy Decoding hoặc Beam Search. Greedy Decoding chọn ký tự có xác suất cao nhất tại mỗi bước, đơn giản và phù hợp cho suy diễn nhanh. Beam Search giữ lại nhiều chuỗi ứng viên tốt nhất tại mỗi bước, nhờ đó có thể tìm được chuỗi dự đoán tốt hơn, nhưng chi phí tính toán cao hơn.

2.5. Các chỉ số đánh giá

Dự án sử dụng ba chỉ số chính: Word Accuracy, Character Error Rate và Normalized Edit Distance.

2.5.1. Word Accuracy

Word Accuracy đo tỷ lệ số từ được dự đoán chính xác hoàn toàn:

$$WA = \frac{\text{Số từ dự đoán đúng hoàn toàn}}{\text{Tổng số từ kiểm thử}} \times 100\%$$

Đây là chỉ số khá nghiêm ngặt, vì chỉ cần sai một ký tự thì cả từ được xem là sai.

2.5.2. Character Error Rate

Character Error Rate (CER) đo tỷ lệ lỗi ở mức ký tự dựa trên edit distance:

$$CER = \frac{\sum \text{edit_distance}(\text{pred}, \text{target})}{\sum \text{len}(\text{target})} \times 100\%$$

CER giúp đánh giá chi tiết hơn Word Accuracy. Một từ có thể bị tính là sai hoàn toàn, nhưng thực tế mô hình chỉ sai một hoặc hai ký tự.

2.5.3. Normalized Edit Distance

Normalized Edit Distance (NED) là chỉ số dựa trên edit distance đã chuẩn hóa theo độ dài chuỗi:

$$NED = 1 - \frac{1}{N} \sum_{i=1}^N \frac{\text{edit_distance}(\text{pred}_i, \text{target}_i)}{\max(\text{len}(\text{pred}_i), \text{len}(\text{target}_i))}$$

NED càng cao thì chuỗi dự đoán càng gần với nhãn đúng. Chỉ số này giúp đánh giá mức độ tương đồng giữa dự đoán và ground truth theo từng mẫu.

2.6. Tổng kết chương

Chương này đã trình bày các cơ sở lý thuyết chính liên quan đến dự án, gồm bài toán nhận dạng chữ viết tay, CNN, ResNet18, BiLSTM, CTC, Attention, Encoder–Decoder, Teacher Forcing, các phương pháp giải mã và các chỉ số đánh giá. Những nội dung này là nền tảng để xây dựng và phân tích hai mô hình được triển khai trong các chương tiếp theo.

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1. Bộ dữ liệu IAM word-level

Dự án sử dụng IAM Handwriting Dataset ở mức word-level. Đây là bộ dữ liệu chữ viết tay tiếng Anh được sử dụng phổ biến trong các nghiên cứu về nhận dạng chữ viết tay ngoại tuyến. Ở mức word-level, mỗi mẫu dữ liệu là một ảnh đã được tách sẵn, chứa một từ viết tay và có nhãn văn bản tương ứng.

Trong phạm vi bài tập lớn, nhóm chọn dữ liệu ở mức từ đơn lẻ thay vì mức dòng hoặc tài liệu hoàn chỉnh. Cách làm này giúp dự án tập trung vào bài toán chính: nhận dạng chuỗi ký tự từ một ảnh chữ viết tay, thay vì phải xử lý thêm các bước như phát hiện vùng chữ, tách dòng, tách từ hoặc phân tích bố cục trang.

Sau quá trình lọc và làm sạch, tổng số mẫu được sử dụng là 96,835. Mỗi mẫu gồm một ảnh chữ viết tay và một nhãn là từ tiếng Anh tương ứng.



Hình 3.1: Một số ảnh mẫu từ IAM Handwriting Dataset ở mức word-level

Dữ liệu chữ viết tay trong IAM có nhiều đặc điểm khiến bài toán khó hơn so với nhận dạng chữ in. Nét chữ giữa các người viết thay đổi khá lớn; một số ký tự có hình dạng gần giống nhau như o và a, r và v, hoặc l và i. Ngoài ra, độ dài từ không cố định, ảnh có thể bị nghiêng, mờ hoặc chứa nhiều khoảng trắng ở hai bên. Những yếu tố này yêu cầu mô hình phải trích xuất đặc trưng ảnh tốt và vẫn giữ được quan hệ tuần tự giữa các ký tự trong từ.

Dữ liệu đầu vào của dự án được tổ chức gồm thư mục ảnh và file nhãn. Thư mục `words/` chứa các ảnh chữ viết tay, còn file `label.txt` lưu thông tin nhãn tương ứng.

```
1 dataset/  
2 +-- words/  
3 |   +-- image_001.png  
4 |   +-- image_002.png  
5 |   +-- image_003.png
```

```

6 |   +-- ...
7 +-- label.txt

```

Mỗi dòng trong file nhãn ánh xạ một ảnh với nội dung văn bản của ảnh đó. Trong quá trình chuẩn bị dữ liệu, hệ thống đọc file nhãn, kiểm tra sự tồn tại của ảnh, xử lý nhãn và loại bỏ các mẫu không hợp lệ.

3.2. Làm sạch nhãn, xây dựng từ vựng và chia tập dữ liệu

Nhãn ban đầu có thể chứa nhiều loại ký tự khác nhau. Để đơn giản hóa bài toán và thống nhất không gian đầu ra, nhóm thực hiện làm sạch nhãn trước khi huấn luyện. Các nhãn được chuyển về chữ thường, loại bỏ ký tự không thuộc phạm vi hỗ trợ và chỉ giữ lại các mẫu hợp lệ sau khi xử lý.

Trong dự án, nhóm tập trung vào các từ tiếng Anh gồm các ký tự chữ cái thường. Việc giới hạn không gian ký tự giúp quá trình huấn luyện ổn định hơn và giảm độ phức tạp của bài toán.

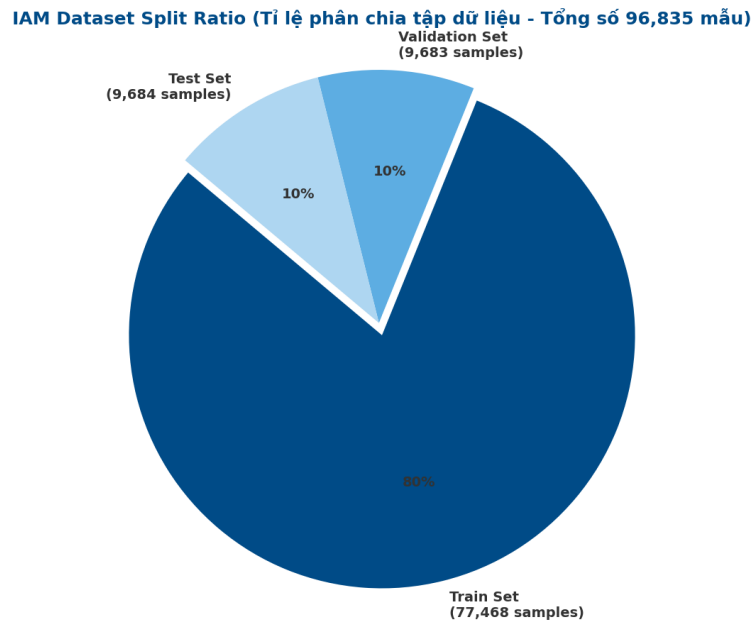
Sau khi làm sạch nhãn, hệ thống xây dựng bộ từ vựng ký tự dùng cho quá trình encode và decode. Mỗi ký tự được ánh xạ thành một chỉ số nguyên. Với mô hình CTC, bộ từ vựng cần thêm blank token để phục vụ CTC Loss, thường được gán chỉ số 0. Với mô hình Attention, hệ thống sử dụng thêm các token đặc biệt như `<pad>`, `<sos>` và `<eos>` để padding, đánh dấu bắt đầu và kết thúc chuỗi.

Sau bước lọc và làm sạch, dữ liệu được chia thành ba tập train, validation và test theo tỉ lệ 80/10/10. Kết quả phân chia được trình bày trong Bảng 3.1.

Bảng 3.1: Phân chia bộ dữ liệu IAM sau tiền xử lý

Tập dữ liệu	Số mẫu	Tỉ lệ
Train Set	77,468	80%
Validation Set	9,683	10%
Test Set	9,684	10%
Tổng cộng	96,835	100%

Tập train được dùng để cập nhật trọng số mô hình. Tập validation dùng để theo dõi quá trình huấn luyện và lựa chọn checkpoint tốt nhất. Tập test chỉ được sử dụng ở bước đánh giá cuối cùng, nhằm phản ánh khách quan hơn hiệu quả của mô hình sau khi huấn luyện.



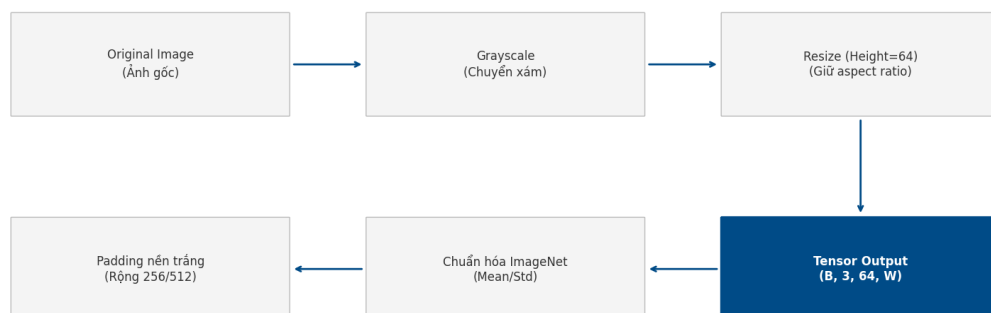
Hình 3.2: Tỉ lệ phân chia dữ liệu train, validation và test

3.3. Tiền xử lý và tăng cường dữ liệu

Tiền xử lý ảnh là bước quan trọng vì chất lượng đầu vào ảnh hưởng trực tiếp đến khả năng học của mô hình. Trong dự án, ảnh được xử lý theo một pipeline thống nhất trước khi đưa vào mạng học sâu.

Ảnh đầu vào được đọc từ đường dẫn tương ứng, chuyển về grayscale nếu cần, sau đó resize về chiều cao cố định. Khi resize, hệ thống giữ nguyên tỉ lệ khía cạnh để tránh làm méo chữ. Nếu ảnh có chiều rộng nhỏ hơn kích thước mục tiêu, hệ thống padding nền trắng về bên phải; nếu ảnh quá rộng, chiều rộng sẽ được giới hạn theo cấu hình. Cuối cùng, ảnh grayscale được nhân bản thành 3 kênh để tương thích với ResNet18, rồi được chuẩn hóa và chuyển thành tensor.

Preprocessing Pipeline (Các bước Tiền xử lý ảnh)



Hình 3.3: Pipeline tiền xử lý ảnh đầu vào

Trong thiết kế thực nghiệm, hai mô hình có thể sử dụng kích thước ảnh đầu vào khác nhau. Với CTC Baseline, ảnh được resize và padding về kích thước 64×256 , sau đó feature map được reshape thành chuỗi có độ dài thời gian $T = 16$. Với Attention Model, ảnh được resize và padding về kích thước 64×512 , tạo ra chuỗi đặc trưng có độ dài $T = 32$.

Trong mã nguồn cài đặt thực tế, hệ thống hỗ trợ cấu hình động thông qua file `config.py`. Cấu hình mặc định có thể dùng chiều rộng 512 để tối ưu hơn cho mô hình Attention, trong khi CTC Baseline vẫn có thể thích ứng nhờ đặc điểm của CNN và LSTM.

Bảng 3.2: Kích thước ảnh đầu vào của hai mô hình

Mô hình	Kích thước ảnh	Feature map sau CNN	Độ dài chuỗi
CTC Baseline	64×256	$(B, 256, 4, 16)$	$T = 16$
Attention Model	64×512	$(B, 256, 4, 32)$	$T = 32$

Do ResNet18 thường được huấn luyện trước trên ImageNet, ảnh đầu vào được chuẩn hóa theo trung bình và độ lệch chuẩn của ImageNet:

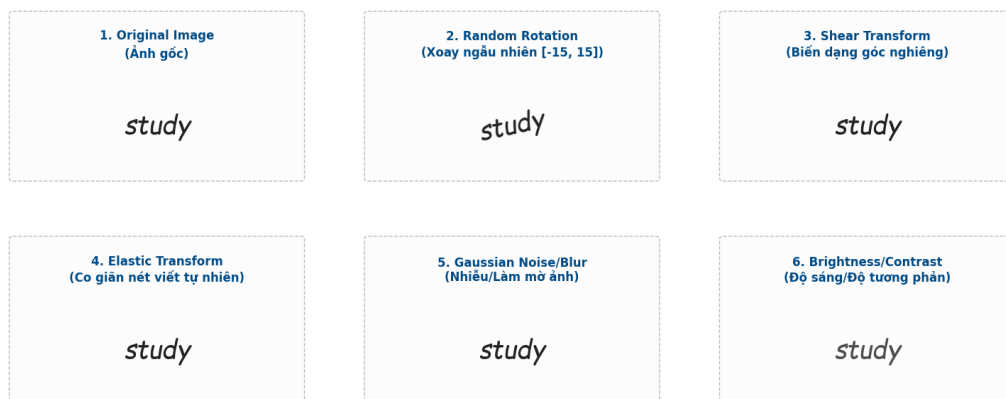
$$\mu = [0.485, 0.456, 0.406]$$

$$\sigma = [0.229, 0.224, 0.225]$$

Mặc dù ảnh chữ viết tay ban đầu là grayscale, việc nhân bản thành 3 kênh giúp tận dụng được encoder ResNet18 mà không cần thay đổi toàn bộ kiến trúc đầu vào.

Để mô hình gặp nhiều biến thể ảnh hơn trong quá trình học, nhóm sử dụng tăng cường dữ liệu cho tập train. Các phép biến đổi gồm xoay ngẫu nhiên trong khoảng $[-15^\circ, 15^\circ]$, affine/shear transform, elastic transform, Gaussian blur, Gaussian noise và thay đổi brightness/contrast. Các phép này mô phỏng một phần sự đa dạng của chữ viết tay và điều kiện ảnh đầu vào.

Data Augmentation Techniques (Các phép tăng cường dữ liệu)



Hình 3.4: Ví dụ minh họa một số phép tăng cường dữ liệu

Tăng cường dữ liệu chỉ được áp dụng cho tập train. Tập validation và test không áp dụng augmentation để quá trình đánh giá phản ánh đúng khả năng của mô hình trên dữ liệu gốc.

3.4. Dataset, DataLoader và pipeline chuẩn bị dữ liệu

Dự án sử dụng `Dataset` và `DataLoader` của PyTorch để quản lý quá trình đọc dữ liệu và tạo batch. `Dataset` chịu trách nhiệm đọc ảnh, áp dụng tiền xử lý, mã hóa nhãn và trả về tensor tương ứng. `DataLoader` gom các mẫu thành batch, shuffle dữ liệu huấn luyện và hỗ trợ quá trình train hiệu quả hơn.

Vì các nhãn có độ dài khác nhau, hệ thống cần xử lý riêng khi tạo batch. Với Attention Model, các chuỗi nhãn được padding bằng token `<pad>` để có cùng độ dài trong batch; khi tính loss, các vị trí padding được bỏ qua bằng `ignore_index`. Với CTC Baseline, ngoài ảnh và nhãn đã encode, mô hình cần thêm độ dài thật của nhãn và độ dài chuỗi đầu vào để CTC Loss tính xác suất trên các alignment hợp lệ.

Toàn bộ quá trình chuẩn bị dữ liệu được thực hiện thông qua script:

```
1 python -m src.prepare_data
```

Script này đọc dữ liệu gốc từ thư mục `dataset/`, đọc file `label.txt`, kiểm tra ảnh tồn tại và có thể đọc được, làm sạch nhãn, lọc các mẫu không hợp lệ, chia dữ liệu thành train/validation/test và lưu các file CSV đã xử lý. Việc tách riêng bước chuẩn bị dữ liệu giúp hệ thống dễ kiểm soát hơn và thuận tiện khi huấn luyện lại mô hình.

3.5. Hạn chế dữ liệu và tổng kết chương

Mặc dù IAM là bộ dữ liệu chuẩn và phù hợp cho bài toán nhận dạng chữ viết tay, dữ liệu này vẫn khác đáng kể so với ảnh thực tế do người dùng tự chụp hoặc tự viết. Ảnh trong IAM thường có chất lượng và cách cắt tương đối chuẩn, trong khi ảnh thực tế có thể có nền phức tạp, ánh sáng không đều, góc chụp lệch hoặc nét chữ cá nhân khác nhiều so với dữ liệu huấn luyện.

Vì vậy, kết quả tốt trên tập test IAM không đảm bảo mô hình sẽ hoạt động ổn định với mọi ảnh chữ viết tay thực tế. Đây là một hạn chế quan trọng cần được lưu ý khi đánh giá khả năng tổng quát hóa của hệ thống.

Chương này đã trình bày bộ dữ liệu được sử dụng trong dự án, cách tổ chức dữ liệu đầu vào, quá trình làm sạch nhãn, xây dựng vocabulary, chia tập dữ liệu, tiền xử lý ảnh, tăng cường dữ liệu và xây dựng `DataLoader`. Đây là nền tảng để huấn luyện và đánh giá hai mô hình CTC Baseline và Attention Model ở các chương tiếp theo.

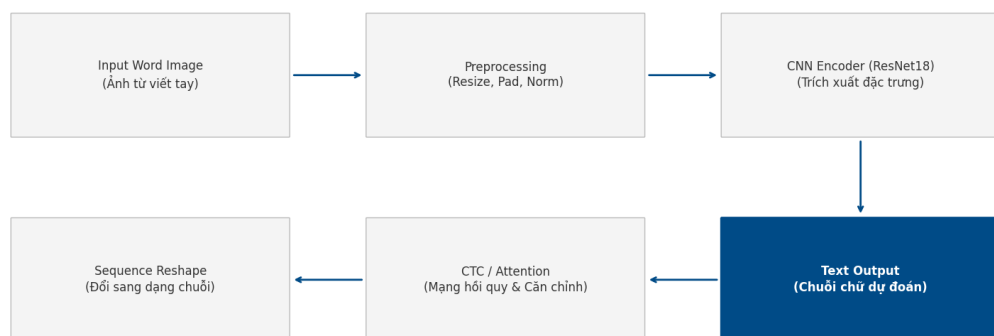
CHƯƠNG 4. KIẾN TRÚC MÔ HÌNH

4.1. Tổng quan hệ thống và xử lý đặc trưng

Dự án xây dựng hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ theo hướng học sâu. Đầu vào của hệ thống là một ảnh chứa duy nhất một từ tiếng Anh viết tay, còn đầu ra là chuỗi ký tự dự đoán tương ứng với nội dung trong ảnh.

Pipeline tổng quát gồm các bước: tiền xử lý ảnh, trích xuất đặc trưng bằng CNN, chuyển feature map thành chuỗi đặc trưng, xử lý chuỗi bằng mô hình tuần tự, sau đó giải mã để tạo chuỗi ký tự đầu ra.

Overall HTR System Pipeline (Sơ đồ luồng xử lý hệ thống)



Hình 4.1: Pipeline tổng quan của hệ thống nhận dạng chữ viết tay

Trong dự án, nhóm triển khai và so sánh hai hướng tiếp cận chính:

- **CTC Baseline:** sử dụng ResNet18 làm CNN Encoder, BiLSTM để xử lý chuỗi đặc trưng và CTC Loss để học cân chỉnh giữa ảnh đầu vào và chuỗi ký tự đầu ra.
- **Attention Model:** sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder để sinh chuỗi ký tự theo cơ chế Encoder–Decoder.

Hai mô hình đều dùng ResNet18 để trích xuất đặc trưng ảnh, nhưng khác nhau ở cách xử lý chuỗi và cách sinh đầu ra. CTC Baseline dự đoán phân phối ký tự theo từng timestep rồi dùng CTC Decoding để rút gọn thành chuỗi cuối cùng. Attention Model sinh từng ký tự thông qua decoder tự hồi quy.

4.1.1. CNN Encoder sử dụng ResNet18

Trong bài toán nhận dạng chữ viết tay, ảnh đầu vào chứa các nét chữ dưới dạng thông tin không gian. Để mô hình xử lý hiệu quả hơn, ảnh cần được chuyển từ pixel thô sang biểu diễn đặc trưng. CNN Encoder đảm nhiệm vai trò này.

Dự án sử dụng ResNet18 làm CNN Encoder. Đây là kiến trúc tương đối gọn, dễ triển khai và có khả năng trích xuất đặc trưng ảnh tốt. Trong hệ thống, ResNet18 không được dùng như một mô hình phân loại ảnh hoàn chỉnh mà chỉ dùng phần trích xuất đặc trưng để tạo feature map.

Với ảnh đầu vào sau tiền xử lý, encoder tạo ra feature map có dạng:

$$F \in \mathbb{R}^{B \times C \times H' \times W'}$$

Trong đó B là batch size, C là số kênh đặc trưng, H' là chiều cao feature map và W' là chiều rộng feature map.

Trong cấu hình thiết kế lý thuyết của nhóm:

- CTC Baseline sử dụng ảnh kích thước 64×256 , tạo feature map xấp xỉ $(B, 256, 4, 16)$.
- Attention Model sử dụng ảnh kích thước 64×512 , tạo feature map xấp xỉ $(B, 256, 4, 32)$.

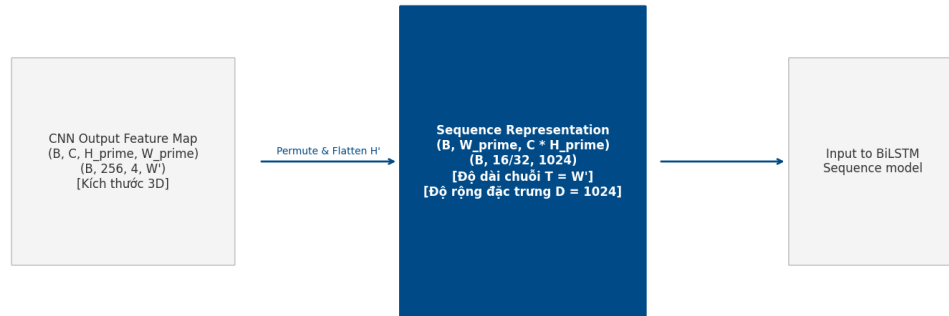
Trong mã nguồn thực tế, do CNN có thể xử lý linh hoạt theo chiều rộng ảnh, cả hai mô hình có thể dùng chung cấu hình chiều rộng mặc định là 512 thông qua file `config.py`. Cách làm này giúp đơn giản hóa pipeline dữ liệu.

4.1.2. Chuyển feature map thành chuỗi đặc trưng

Sau khi đi qua ResNet18, feature map vẫn là biểu diễn hai chiều. Tuy nhiên, nhận dạng chữ viết tay là bài toán sinh chuỗi ký tự, nên feature map cần được chuyển thành chuỗi đặc trưng theo chiều ngang của ảnh.

Vì chữ tiếng Anh thường được viết và đọc từ trái sang phải, chiều rộng của feature map có thể xem như trục thời gian. Mỗi vị trí theo chiều ngang tương ứng với một vùng đặc trưng của ảnh.

Feature Reshape Process (Quy trình biến đổi đặc trưng không gian thành chuỗi)



Hình 4.2: Minh họa quá trình chuyển feature map 2D thành chuỗi đặc trưng

Với CTC Baseline, quá trình reshape có dạng:

$$(B, 256, 4, 16) \rightarrow (B, 16, 1024)$$

Trong đó chuỗi đầu ra có độ dài $T = 16$, mỗi timestep có vector đặc trưng 1024 chiều.

Với Attention Model, quá trình reshape có dạng:

$$(B, 256, 4, 32) \rightarrow (B, 32, 1024)$$

Chuỗi dài hơn giúp cơ chế attention có nhiều vị trí đặc trưng hơn để tập trung khi sinh từng ký tự.

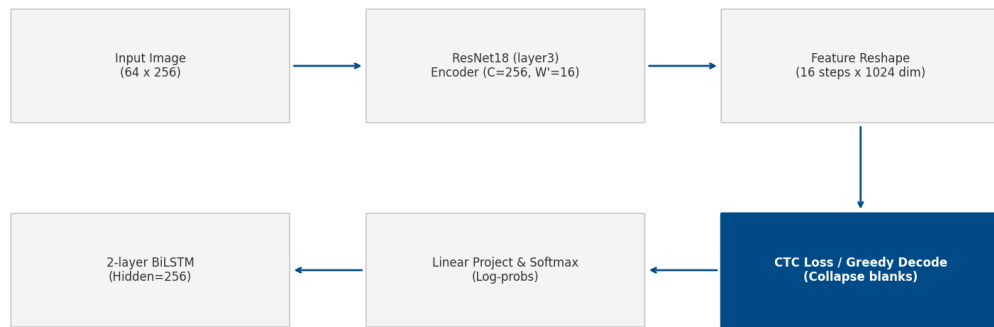
4.2. Mô hình CTC Baseline

CTC Baseline là mô hình cơ sở của dự án, được xây dựng để tạo mốc so sánh với Attention Model. Kiến trúc này đơn giản, trực tiếp và phù hợp với bài toán nhận dạng chuỗi khi không có thông tin căn chỉnh cụ thể giữa ảnh và từng ký tự.

Pipeline của mô hình CTC Baseline gồm:

Image $\rightarrow$ ResNet18 Encoder $\rightarrow$ Sequence Reshape $\rightarrow$ BiLSTM $\rightarrow$ Linear Projection $\rightarrow$ CTC Loss

CTC Baseline Architecture (Kiến trúc mô hình CTC)



Hình 4.3: Kiến trúc mô hình CTC Baseline

Sau khi feature map được chuyển thành chuỗi đặc trưng, chuỗi này được đưa qua BiLSTM. BiLSTM xử lý dữ liệu theo cả hai chiều trái sang phải và phải sang trái, nhờ đó mỗi vị trí trong chuỗi đầu ra có thêm ngữ cảnh từ các vùng ảnh xung quanh. Trong mô hình CTC Baseline, BiLSTM gồm 2 lớp với hidden dimension bằng 256 cho mỗi chiều, nên vector đầu ra tại mỗi timestep có kích thước 512.

Đầu ra của BiLSTM được đưa qua một lớp tuyến tính để ánh xạ sang không gian ký tự. Nếu vocabulary có V ký tự, đầu ra tại mỗi timestep là một vector kích thước V . Sau đó, mô hình áp dụng `log_softmax` để tạo phân phối xác suất logarit trên các ký tự trước khi đưa vào CTC Loss.

CTC Loss cho phép huấn luyện mô hình mà không cần biết vị trí chính xác của từng ký tự trong ảnh. Với mỗi ảnh đầu vào, mô hình sinh ra một chuỗi phân phối xác suất theo số timestep, trong khi nhãn thật là chuỗi ký tự có độ dài thay đổi. CTC xem xét tất cả các căn chỉnh hợp lệ có thể rút gọn về nhãn đúng, rồi tối đa hóa tổng xác suất của các căn chỉnh đó.

Khi suy diễn, mô hình sử dụng CTC Greedy Decoding. Quá trình này chọn ký tự có xác suất cao nhất tại mỗi timestep, sau đó gộp các ký tự giống nhau đứng liên tiếp và loại bỏ blank token.

Ví dụ, với blank token được biểu diễn là `-`, chuỗi thô:

-hhe-ll-oo-

sau khi xử lý có thể trở thành:

helo

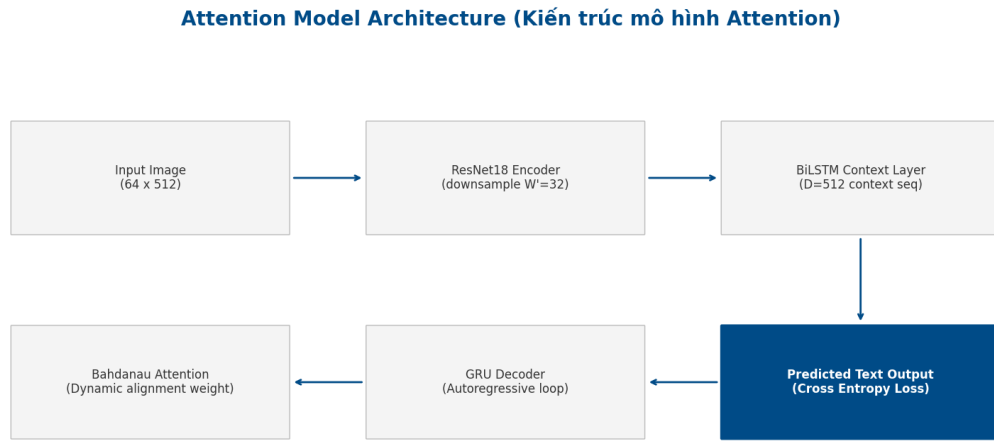
Phương pháp này nhanh, dễ triển khai và phù hợp để đánh giá baseline. Tuy nhiên, do chỉ chọn phương án tốt nhất tại từng timestep, greedy decoding có thể bỏ qua những chuỗi có xác suất tổng thể tốt hơn.

4.3. Mô hình Attention

Attention Model là kiến trúc chính của dự án, được xây dựng theo hướng Encoder–Decoder. Khác với CTC Baseline, mô hình này sử dụng decoder tự hồi quy để sinh chuỗi ký tự từng bước.

Pipeline của mô hình Attention gồm:

Image → ResNet18 Encoder → Sequence Reshape → BiLSTM Context Encoder → Bahdanau Attention → GRU Decoder



Hình 4.4: Kiến trúc mô hình Attention

4.3.1. BiLSTM Context Encoder

Sau khi ResNet18 trích xuất feature map và feature map được reshape thành chuỗi, mô hình đưa chuỗi này qua BiLSTM Context Encoder. Khối này bổ sung thông tin ngữ cảnh hai chiều cho từng vị trí đặc trưng.

Nếu chỉ sử dụng đặc trưng CNN cục bộ, mỗi vị trí trong chuỗi có thể thiếu thông tin về các ký tự xung quanh. Điều này khiến attention khó xác định vùng cần tập trung ở từng bước giải mã, nhất là với chữ viết liền nét hoặc các ký tự có hình dạng gần giống nhau. BiLSTM Context Encoder giúp mỗi vector đặc trưng chứa thêm thông tin từ cả bên trái và bên phải của từ viết tay.

4.3.2. Bahdanau Attention

Bahdanau Attention được dùng để tính trọng số chú ý giữa trạng thái hiện tại của decoder và các vị trí đặc trưng từ encoder. Tại mỗi bước giải mã t , mô hình tính điểm attention giữa trạng thái ẩn trước đó của decoder s_{t-1} và từng vector encoder h_s :

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Các điểm này được chuẩn hóa bằng softmax:

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{k=1}^T \exp(e_{t,k})}$$

Vector ngữ cảnh tại bước t được tính bằng tổng có trọng số:

$$c_t = \sum_{s=1}^T \alpha_{t,s} h_s$$

Trong đó, $\alpha_{t,s}$ thể hiện mức độ mô hình tập trung vào vị trí s của chuỗi đặc trưng khi sinh ký tự tại bước t . Cơ chế này giúp mô hình học căn chỉnh mềm giữa vùng ảnh và ký tự đầu ra.

4.3.3. GRU Decoder và Teacher Forcing

Decoder của mô hình Attention sử dụng GRUCell. Tại mỗi bước giải mã, decoder nhận embedding của ký tự trước đó và vector ngữ cảnh c_t từ attention, sau đó cập nhật trạng thái ẩn và dự đoán ký tự tiếp theo.

Quá trình sinh chuỗi bắt đầu từ token $\langle \text{sos} \rangle$ và kết thúc khi mô hình sinh token $\langle \text{eos} \rangle$ hoặc đạt độ dài tối đa cho phép. Vì decoder là mô hình tự hồi quy, dự đoán hiện tại phụ thuộc vào các ký tự đã sinh trước đó. Điều này giúp mô hình khai thác một phần quan hệ giữa các ký tự trong từ.

Trong quá trình huấn luyện, mô hình sử dụng Teacher Forcing. Thay vì luôn dùng ký tự do mô hình dự đoán ở bước trước, Teacher Forcing cho phép đưa ký tự đúng từ ground truth vào decoder ở một số bước. Kỹ thuật này giúp quá trình học ổn định hơn, đặc biệt ở giai đoạn đầu khi mô hình còn dự đoán sai nhiều. Khi validation, test hoặc predict ảnh thực tế, Teacher Forcing không được sử dụng.

4.3.4. Greedy Decoding và Beam Search

Khi suy diễn, Attention Model có thể dùng Greedy Decoding hoặc Beam Search. Với Greedy Decoding, tại mỗi bước, decoder chọn ký tự có xác suất cao nhất làm đầu ra và đưa ký tự đó vào bước tiếp theo. Phương pháp này nhanh và phù hợp khi cần dự đoán trực tiếp trên ảnh đơn.

Beam Search giữ lại nhiều chuỗi ứng viên có xác suất cao nhất tại mỗi bước thay vì chỉ giữ một chuỗi duy nhất. Cách này giúp mô hình xem xét nhiều khả năng hơn trong quá trình sinh chuỗi, nhưng chi phí tính toán cao hơn.

4.4. So sánh hai mô hình và tổ chức suy diễn

Bảng 4.1 trình bày so sánh tổng quan giữa CTC Baseline và Attention Model.

Bảng 4.1: So sánh kiến trúc CTC Baseline và Attention Model

Tiêu chí	CTC Baseline	Attention Model
CNN Encoder	ResNet18	ResNet18
Sequence Modeling	2-layer BiLSTM	2-layer BiLSTM Context Encoder
Cơ chế căn chỉnh	CTC alignment ngầm	Bahdanau soft attention
Decoder	Không có decoder tự hồi quy	GRU Decoder
Loss Function	CTC Loss	Cross Entropy Loss
Giải mã	CTC Greedy Decoding	Greedy Decoding hoặc Beam Search
Kích thước ảnh đầu vào	64×256	64×512
Độ dài chuỗi đặc trưng	$T = 16$	$T = 32$
Ưu điểm	Đơn giản, ổn định, dễ huấn luyện	Linh hoạt hơn, có thể trực quan hóa attention map
Hạn chế	Bị giới hạn bởi số timestep và giả định độc lập có điều kiện	Phức tạp hơn, huấn luyện chậm hơn, có thể bị lệch attention

Về tổng thể, CTC Baseline đơn giản hơn và dễ huấn luyện hơn. Mô hình này phù hợp để làm baseline vì không cần decoder tự hồi quy và không cần attention. Ngược lại, Attention Model phức tạp hơn nhưng linh hoạt hơn, do có thể sinh chuỗi từng bước và trực quan hóa attention map.

Trong mã nguồn, các thuật toán giải mã được tách riêng trong thư mục `src/inference`. Đây là cách tổ chức hợp lý vì mô hình học sâu không trực tiếp sinh ra chuỗi văn bản hoàn chỉnh, mà thường trả về tensor logits hoặc phân phối xác suất trên vocabulary. Phần inference chịu trách nhiệm chuyển các đầu ra này thành chuỗi ký tự cuối cùng.

Việc tách riêng inference giúp cấu trúc mã nguồn rõ ràng hơn, tránh lặp code giữa các script đánh giá và dự đoán, đồng thời dễ thay thế hoặc nâng cấp thuật toán giải mã về sau.

4.5. Tổng kết chương

Chương này đã trình bày hai kiến trúc mô hình được sử dụng trong dự án. CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss để nhận dạng chuỗi ký tự mà không cần thông tin căn chỉnh vị trí ký tự. Attention Model sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder để sinh chuỗi ký tự từng bước.

Hai mô hình có cùng mục tiêu nhận dạng từ viết tay nhưng khác nhau về cách căn chỉnh, cách huấn luyện và cách giải mã. Việc triển khai cả hai mô hình giúp nhóm có cơ sở so sánh hướng tiếp cận CTC và hướng tiếp cận Attention trong bài toán word-level HTR.

CHƯƠNG 5. CÀI ĐẶT HỆ THỐNG

5.1. Môi trường phát triển

Dự án được xây dựng bằng ngôn ngữ Python và framework PyTorch. Toàn bộ quá trình huấn luyện, đánh giá và suy diễn được thực hiện trên môi trường local, không sử dụng các dịch vụ OCR API bên thứ ba.

Việc huấn luyện local giúp nhóm chủ động kiểm soát dữ liệu, mô hình, checkpoint và quá trình đánh giá. Đồng thời, điều này cũng giúp dự án thể hiện rõ quy trình xây dựng một hệ thống nhận dạng chữ viết tay từ đầu, thay vì chỉ gọi lại một mô hình hoặc dịch vụ có sẵn.

Bảng 5.1 trình bày môi trường phát triển chính của dự án.

Bảng 5.1: Môi trường phát triển của dự án

Thành phần	Thông tin
Ngôn ngữ lập trình	Python 3.11
Framework học sâu	PyTorch
Xử lý ảnh	OpenCV, Pillow
Xử lý dữ liệu	NumPy, Pandas
Tăng cường dữ liệu	Albumentations
Tính edit distance	editdistance
Trực quan hóa	Matplotlib
Môi trường chạy	Local machine

5.2. Phần cứng sử dụng

Trong quá trình huấn luyện, dự án sử dụng GPU để tăng tốc tính toán. Việc sử dụng GPU là cần thiết vì cả hai mô hình đều có CNN Encoder và các khối xử lý chuỗi, khiến thời gian huấn luyện trên CPU trở nên rất lâu.

Bảng 5.2 trình bày thông tin phần cứng chính được sử dụng.

Bảng 5.2: Phần cứng sử dụng trong quá trình huấn luyện

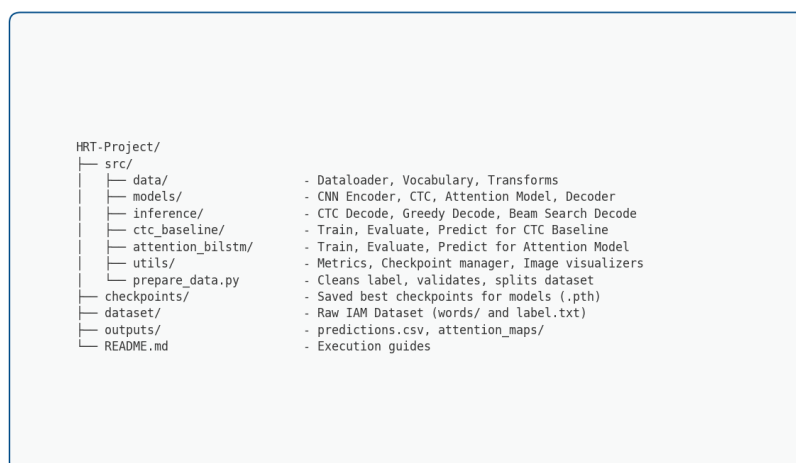
Thành phần	Thông tin
GPU	NVIDIA GeForce RTX 5060 Ti
VRAM	16 GB
Driver Version	581.80
CUDA Support	CUDA 13.0

Cấu hình này cho phép huấn luyện và đánh giá mô hình trên tập IAM ở mức word-level với batch size phù hợp, đồng thời lưu checkpoint tốt nhất trong quá trình train.

5.3. Cấu trúc thư mục dự án

Mã nguồn dự án được tổ chức theo hướng tách rõ các thành phần: dữ liệu, mô hình, thuật toán giải mã, tiện ích, pipeline huấn luyện và pipeline suy diễn. Cấu trúc thư mục chính được trình bày dưới đây.

Codebase Directory Structure (Cấu trúc thư mục dự án)



5.4. Vai trò của các thư mục chính

5.4.1. Thư mục src/data

Thư mục `src/data` chứa các thành phần phục vụ đọc dữ liệu, xử lý nhãn, transform ảnh và xây dựng DataLoader. Đây là phần trung gian giữa dữ liệu thô và mô hình học sâu.

Các nhiệm vụ chính của nhóm file trong thư mục này gồm:

- Đọc danh sách ảnh và nhãn từ file CSV đã xử lý.
- Áp dụng tiền xử lý ảnh.
- Áp dụng tăng cường dữ liệu cho tập train.
- Encode nhãn thành tensor số.
- Padding nhãn trong batch nếu cần.
- Trả về dữ liệu theo định dạng phù hợp với từng mô hình.

5.4.2. Thư mục src/models

Thư mục `src/models` chứa các định nghĩa kiến trúc mô hình. Mỗi file đảm nhiệm một thành phần riêng:

- `resnet_encoder.py`: định nghĩa CNN Encoder sử dụng ResNet18.
- `ctc_baseline.py`: định nghĩa mô hình CTC Baseline.

- `attention.py`: định nghĩa Bahdanau Attention.
- `gru_decoder.py`: định nghĩa GRU Decoder.
- `attention_model.py`: kết hợp encoder, attention và decoder thành mô hình hoàn chỉnh.

Cách tách file này giúp mã nguồn dễ đọc hơn. Khi cần chỉnh sửa một thành phần, ví dụ attention hoặc decoder, nhóm có thể chỉnh trực tiếp file tương ứng mà không ảnh hưởng đến toàn bộ hệ thống.

5.4.3. Thư mục `src/inference`

Thư mục `src/inference` chứa các thuật toán giải mã. Đây là phần chuyển đổi đầu ra dạng tensor của mô hình thành chuỗi ký tự có thể đọc được.

Các file chính gồm:

- `ctc_decode.py`: triển khai CTC Greedy Decoding.
- `greedy_decode.py`: triển khai Greedy Decoding cho Attention Model.
- `beam_search.py`: triển khai Beam Search cho Attention Model.

Việc tách riêng inference giúp tránh trùng lặp code giữa `evaluate.py` và `predict.py`. Đồng thời, nó giúp hệ thống dễ mở rộng nếu muốn bổ sung các phương pháp giải mã khác trong tương lai.

5.4.4. Thư mục `src/utlis`

Thư mục `src/utlis` chứa các hàm tiện ích dùng chung trong toàn dự án. Các chức năng thường gặp gồm:

- Tính metric như Word Accuracy, CER và NED.
- Lưu và load checkpoint.
- Thiết lập seed để tăng tính tái lập.
- Xử lý đường dẫn file.
- Trực quan hóa attention map.

Việc gom các hàm dùng chung vào `utlis` giúp giảm trùng lặp code và làm cho các script train, evaluate, predict gọn hơn.

5.4.5. Thư mục `src/ctc_baseline`

Thư mục `src/ctc_baseline` chứa pipeline riêng cho mô hình CTC Baseline. Cụ thể:

- `train.py`: huấn luyện CTC Baseline.
- `evaluate.py`: đánh giá mô hình trên test set.
- `predict.py`: dự đoán ảnh đơn hoặc thư mục ảnh.

5.4.6. Thư mục `src/attention_bilstm`

Thư mục `src/attention_bilstm` chứa pipeline riêng cho mô hình Attention. Cụ thể:

- `train.py`: huấn luyện Attention Model.
- `evaluate.py`: đánh giá mô hình trên test set.
- `predict.py`: dự đoán ảnh đơn hoặc thư mục ảnh, hỗ trợ tùy chọn lưu attention map và Beam Search.

5.5. Pipeline chuẩn bị dữ liệu

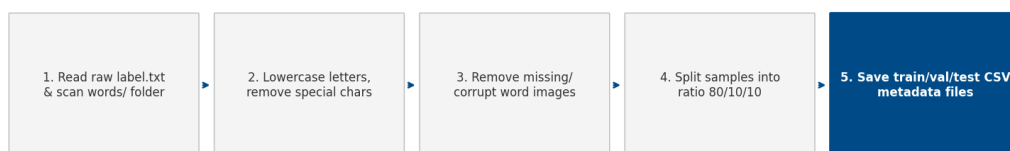
Trước khi huấn luyện, dữ liệu IAM cần được xử lý và chia thành các tập train, validation và test. Quá trình này được thực hiện bằng script:

```
python -m src.prepare_data
```

Script `prepare_data.py` thực hiện các bước chính:

1. Đọc dữ liệu từ thư mục `dataset/`.
2. Đọc file nhãn `label.txt`.
3. Kiểm tra sự tồn tại và tính hợp lệ của ảnh.
4. Làm sạch nhãn.
5. Lọc bỏ các mẫu không hợp lệ.
6. Chia dữ liệu thành train, validation và test.
7. Lưu các file CSV đã xử lý vào thư mục dữ liệu trung gian.

Data Preparation Process Flow (Quy trình chuẩn bị dữ liệu)



Hình 5.2: Luồng chuẩn bị dữ liệu trước khi huấn luyện

Việc tách riêng bước chuẩn bị dữ liệu giúp quá trình huấn luyện tái lập hơn. Sau khi dữ liệu đã được xử lý, các lần huấn luyện sau chỉ cần đọc lại các file CSV thay vì xử lý lại toàn bộ dữ liệu gốc.

5.6. Pipeline huấn luyện

Dự án có hai pipeline huấn luyện riêng cho hai mô hình. Việc tách pipeline giúp cấu hình, loss function và quá trình giải mã của từng mô hình được quản lý rõ ràng.

5.6.1. Huấn luyện CTC Baseline

Mô hình CTC Baseline được huấn luyện bằng lệnh:

```
python -m src.ctc_baseline.train
```

Trong quá trình huấn luyện, ảnh được đưa qua ResNet18 Encoder, reshape thành chuỗi đặc trưng, xử lý bằng BiLSTM và chiếu sang vocabulary. Đầu ra log probability sau đó được đưa vào CTC Loss.

Quy trình huấn luyện CTC Baseline gồm:

1. Load dữ liệu train và validation.
2. Khởi tạo mô hình CTC Baseline.
3. Khởi tạo optimizer và loss function.
4. Lặp qua từng epoch.
5. Tính loss trên tập train.
6. Đánh giá trên tập validation.
7. Lưu checkpoint nếu kết quả validation tốt hơn.

Checkpoint tốt nhất được lưu tại:

```
checkpoints/ctc_baseline/best_ctc_baseline.pth
```

5.6.2. Huấn luyện Attention Model

Mô hình Attention được huấn luyện bằng lệnh:

```
python -m src.attention_bilstm.train
```

Trong quá trình huấn luyện, ảnh được đưa qua ResNet18 Encoder, reshape thành chuỗi đặc trưng và xử lý bởi BiLSTM Context Encoder. Sau đó, decoder sử dụng Bahdanau Attention để sinh chuỗi ký tự từng bước. Loss được tính bằng Cross Entropy Loss, bỏ qua các vị trí padding.

Quy trình huấn luyện Attention Model gồm:

1. Load dữ liệu train và validation.
2. Khởi tạo ResNet18 Encoder, BiLSTM Context Encoder, Attention và GRU Decoder.
3. Encode nhân với các token đặc biệt như <sos>, <eos> và <pad>.

4. Sử dụng Teacher Forcing trong quá trình train.
5. Tính Cross Entropy Loss tại từng bước decoder.
6. Đánh giá trên validation set với Teacher Forcing bằng 0.
7. Lưu checkpoint tốt nhất.

Checkpoint tốt nhất được lưu tại:

```
checkpoints/attention_bilstm/best_attention_model.pth
```

5.7. Chiến lược lưu checkpoint

Trong quá trình huấn luyện, checkpoint được lưu khi mô hình đạt kết quả tốt hơn trên tập validation. Cách làm này giúp tránh việc sử dụng mô hình ở epoch cuối nếu mô hình đã bắt đầu overfit.

Mỗi checkpoint thường chứa:

- Trọng số mô hình.
- Trạng thái optimizer nếu cần tiếp tục huấn luyện.
- Thông tin epoch.
- Kết quả validation tốt nhất tại thời điểm lưu.
- Cấu hình vocabulary hoặc thông tin cần thiết để decode.

Việc lưu checkpoint giúp quá trình đánh giá và dự đoán ảnh thực tế không cần huấn luyện lại từ đầu.

5.8. Pipeline đánh giá

Sau khi huấn luyện, mô hình được đánh giá trên tập test độc lập. Tập test không được sử dụng trong quá trình cập nhật trọng số hoặc lựa chọn mô hình, do đó kết quả trên tập test phản ánh khách quan hơn khả năng của mô hình.

5.8.1. Đánh giá CTC Baseline

Lệnh đánh giá CTC Baseline:

```
python -m src.ctc_baseline.evaluate --checkpoint checkpoints/ctc_baseline/  
best_ctc_baseline.pth
```

Script đánh giá thực hiện các bước:

1. Load checkpoint đã huấn luyện.
2. Load test set.
3. Chạy suy diễn trên từng batch ảnh.
4. Áp dụng CTC Greedy Decoding.

5. So sánh dự đoán với nhãn thật.
6. Tính Word Accuracy, CER và NED.
7. Lưu dự đoán chi tiết ra file CSV.

5.8.2. Đánh giá Attention Model

Lệnh đánh giá Attention Model:

```
python -m src.attention_bilstm.evaluate --checkpoint checkpoints/  
attention_bilstm/best_attention_model.pth
```

Script đánh giá Attention Model thực hiện các bước tương tự, nhưng sử dụng Greedy Decoding hoặc Beam Search tùy cấu hình. Trong quá trình đánh giá, Teacher Forcing không được sử dụng. Mô hình phải tự sinh chuỗi ký tự đầu ra dựa trên các dự đoán trước đó.

5.9. Pipeline dự đoán ảnh thực tế

Ngoài đánh giá trên test set, hệ thống hỗ trợ dự đoán ảnh đơn lẻ hoặc toàn bộ thư mục ảnh. Chức năng này giúp kiểm thử mô hình với ảnh thực tế bên ngoài dataset.

5.9.1. Dự đoán bằng CTC Baseline

Dự đoán một ảnh đơn:

```
python -m src.ctc_baseline.predict --image duong_dan_anh.png
```

Dự đoán toàn bộ thư mục ảnh:

```
python -m src.ctc_baseline.predict --image duong_dan_thu_muc/
```

Với mô hình CTC, ảnh được tiền xử lý, đưa qua mô hình, sau đó kết quả được giải mã bằng CTC Greedy Decoding.

5.9.2. Dự đoán bằng Attention Model

Dự đoán một ảnh đơn bằng Greedy Decoding:

```
python -m src.attention_bilstm.predict --image duong_dan_anh.png
```

Dự đoán một ảnh đơn và lưu attention map:

```
python -m src.attention_bilstm.predict --image duong_dan_anh.png --  
save_attention
```

Dự đoán bằng Beam Search:

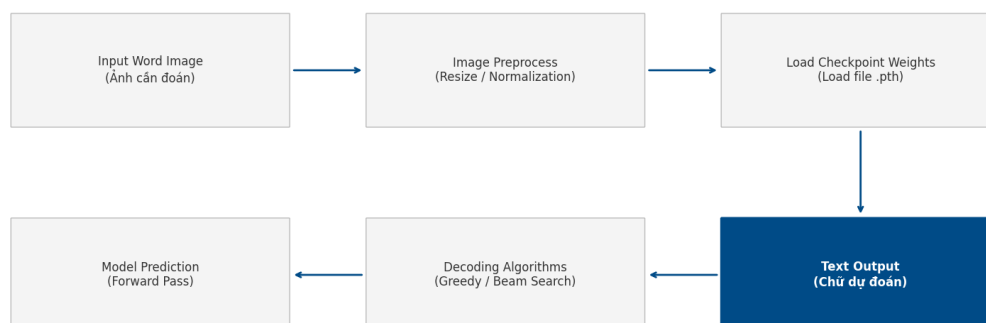
```
python -m src.attention_bilstm.predict --image duong_dan_anh.png --beam
```

Dự đoán toàn bộ thư mục ảnh:

```
python -m src.attention_bilstm.predict --image duong_dan_thu_muc/
```


Tùy chọn `-save_attention` cho phép lưu attention map vào thư mục `outputs/attention_maps/`. Kết quả này hỗ trợ phân tích vùng ảnh mà mô hình tập trung trong quá trình dự đoán.

Inference Pipeline (Quy trình suy diễn của mô hình)



Hình 5.3: Pipeline suy diễn trên ảnh thực tế

5.10. Đầu ra của hệ thống

Trong quá trình đánh giá và dự đoán, hệ thống tạo ra các đầu ra chính sau:

- Chuỗi ký tự dự đoán cho từng ảnh đầu vào.
- File CSV lưu dự đoán chi tiết trên test set.
- Các chỉ số đánh giá như Word Accuracy, CER và NED.
- Attention map đối với mô hình Attention khi sử dụng tùy chọn `-save_attention`.
- Checkpoint mô hình tốt nhất trong quá trình huấn luyện.

Ví dụ, kết quả dự đoán chi tiết có thể được lưu dưới dạng:

```

1 outputs/predictions_ctc.csv
2 outputs/predictions.csv
3 outputs/attention_maps/
  
```

Các file này giúp nhóm kiểm tra lại từng trường hợp dự đoán đúng, dự đoán sai và phân tích nguyên nhân lỗi của mô hình.

5.11. Quản lý phụ thuộc

Danh sách thư viện cần thiết được lưu trong file:

```

1 requirements.txt
  
```

Người dùng có thể cài đặt các thư viện bằng lệnh:

```
python -m pip install -r requirements.txt
```

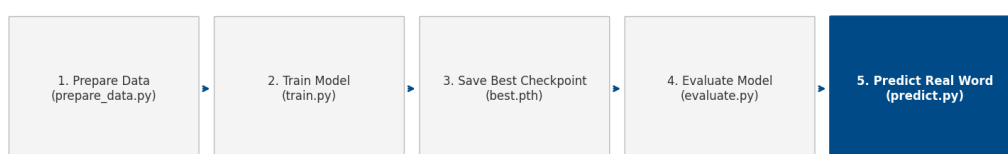
Việc quản lý phụ thuộc bằng `requirements.txt` giúp môi trường chạy dự án dễ tái lập hơn trên máy khác.

5.12. Luồng chạy hệ thống

Luồng chạy đầy đủ của dự án có thể được tóm tắt như sau:

1. Cài đặt môi trường Python và các thư viện cần thiết.
2. Tải IAM Dataset và đặt vào thư mục `dataset/`.
3. Chạy `src.prepare_data` để xử lý dữ liệu.
4. Huấn luyện CTC Baseline hoặc Attention Model.
5. Lưu checkpoint tốt nhất.
6. Đánh giá mô hình trên test set.
7. Dự đoán ảnh thực tế nếu cần.
8. Phân tích kết quả dự đoán và attention map.

End-to-End System Execution Flow (Luồng chạy toàn cục hệ thống)



Hình 5.4: Luồng chạy tổng quát của hệ thống

5.13. Tính mô-đun của hệ thống

Một điểm quan trọng trong cài đặt hệ thống là tính mô-đun. Các thành phần của dự án được tách riêng theo chức năng, giúp dễ phát triển và kiểm thử.

Ví dụ, nếu muốn thay ResNet18 bằng một encoder khác, nhóm có thể chỉnh trong phần `models` mà không cần thay đổi toàn bộ pipeline huấn luyện. Nếu muốn thử một thuật toán giải mã khác, nhóm có thể bổ sung file mới trong `inference`. Nếu muốn thay đổi preprocessing, nhóm có thể chỉnh trong phần `data` hoặc `utils`.

Cách tổ chức này giúp dự án dễ bảo trì hơn, đồng thời thuận lợi cho việc mở rộng trong tương lai.

5.14. Một số lưu ý khi triển khai

Khi chạy dự án trên máy local, cần lưu ý một số điểm:

- Dataset IAM có kích thước lớn, cần đặt đúng cấu trúc thư mục trước khi chạy.
- Checkpoint của mô hình Attention có dung lượng lớn, cần đặt đúng đường dẫn trước khi evaluate hoặc predict.
- Phiên bản PyTorch và CUDA cần tương thích với GPU.
- Ảnh thực tế nên được crop tương đối sát vùng chữ để mô hình dự đoán tốt hơn.
- Với ảnh có nền phức tạp hoặc ánh sáng không đều, kết quả dự đoán có thể kém hơn so với ảnh trong IAM.

5.15. Tổng kết chương

Chương này đã trình bày cách cài đặt hệ thống, bao gồm môi trường phát triển, phần cứng sử dụng, cấu trúc thư mục, vai trò của từng module, pipeline chuẩn bị dữ liệu, huấn luyện, đánh giá và dự đoán ảnh thực tế. Hệ thống được tổ chức theo hướng mô-đun, tách rõ dữ liệu, mô hình, inference và tiện ích, giúp quá trình phát triển, kiểm thử và mở rộng trở nên thuận tiện hơn.

CHƯƠNG 6. THỰC NGHIỆM VÀ KẾT QUẢ

6.1. Thiết lập thực nghiệm

Chương này trình bày quá trình đánh giá hai mô hình nhận dạng chữ viết tay mức độ từ đơn lẻ: CTC Baseline và Attention Model. Hai mô hình được huấn luyện và kiểm thử trên cùng bộ dữ liệu IAM word-level sau tiền xử lý, sử dụng cùng các chỉ số đánh giá để việc so sánh được nhất quán.

Dữ liệu sử dụng trong thực nghiệm là IAM Handwriting Dataset ở mức word-level. Sau khi lọc bỏ các mẫu không hợp lệ và làm sạch nhãn, tổng số mẫu còn lại là 96,835. Dữ liệu được chia thành ba tập train, validation và test theo tỉ lệ 80/10/10.

Bảng 6.1: Phân chia dữ liệu trong thực nghiệm

Tập dữ liệu	Số mẫu	Tỉ lệ
Train Set	77,468	80%
Validation Set	9,683	10%
Test Set	9,684	10%
Tổng cộng	96,835	100%

Tập train được dùng để cập nhật trọng số mô hình, tập validation dùng để theo dõi quá trình huấn luyện và chọn checkpoint tốt nhất, còn tập test dùng cho đánh giá cuối cùng.

Hai mô hình được so sánh trong cùng điều kiện dữ liệu:

- **CTC Baseline:** ResNet18 + BiLSTM + CTC Loss.
- **Attention Model:** ResNet18 + BiLSTM Context Encoder + Bahdanau Attention + GRU Decoder.

Bảng 6.2: Tóm tắt thiết lập thực nghiệm

Thành phần	Thiết lập
Bộ dữ liệu	IAM Handwriting Dataset word-level
Tổng số mẫu	96,835
Tập kiểm thử	9,684 mẫu
Baseline	ResNet18 + BiLSTM + CTC Loss
Mô hình Attention	ResNet18 + BiLSTM + Bahdanau Attention + GRU Decoder
Chỉ số đánh giá	Word Accuracy, CER, NED
Môi trường	Local machine sử dụng PyTorch

Dự án sử dụng ba chỉ số chính: Word Accuracy, Character Error Rate và Normalized Edit Distance. Word Accuracy đo tỷ lệ từ được dự đoán đúng hoàn toàn, CER đo mức độ sai khác ở cấp độ ký tự, còn NED cho biết mức độ tương đồng giữa chuỗi dự đoán và nhãn đúng sau khi chuẩn hóa theo độ dài chuỗi.

6.2. Kết quả trên tập kiểm thử IAM

Kết quả đánh giá trên 9,684 mẫu kiểm thử được trình bày trong Bảng 6.3.

Bảng 6.3: Kết quả so sánh hai mô hình trên IAM Test Set

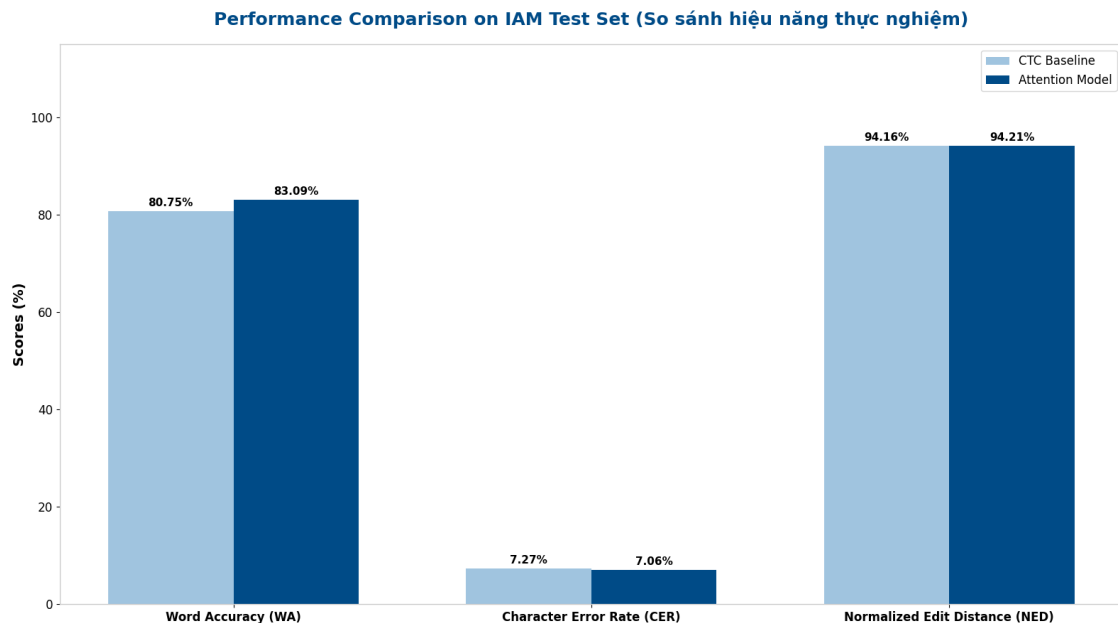
Mô hình	Kiến trúc	Word Accuracy	CER
CTC Baseline	ResNet18 + BiLSTM + CTC Loss	80.75%	7.27%
Attention Model	ResNet18 + BiLSTM + Bahdanau Attention + GRU Decoder	83.09%	7.06%

Bảng 6.4 trình bày thêm chỉ số NED và dung lượng checkpoint của hai mô hình.

Bảng 6.4: Kết quả chi tiết của hai mô hình

Chỉ số	CTC Baseline	Attention Model
Word Accuracy	80.75%	83.09%
Character Error Rate	7.27%	7.06%
Normalized Edit Distance	94.16%	94.21%
Dung lượng checkpoint	khoảng 75 MB	khoảng 91 MB

Attention Model đạt Word Accuracy 83.09%, cao hơn 2.34 điểm phần trăm so với CTC Baseline. Về CER, Attention Model đạt 7.06%, thấp hơn nhẹ so với 7.27% của CTC Baseline. Chỉ số NED của hai mô hình khá gần nhau, lần lượt là 94.16% và 94.21%.



Hình 6.1: Biểu đồ so sánh kết quả CTC Baseline và Attention Model

6.3. Phân tích kết quả

Attention Model có Word Accuracy cao hơn, cho thấy mô hình dự đoán đúng hoàn toàn nhiều từ hơn trên IAM Test Set. Kết quả này có thể đến từ cơ chế attention, vì decoder có khả năng học căn chỉnh mềm giữa đặc trưng ảnh và chuỗi ký tự đầu ra.

Tuy nhiên, mức cải thiện không quá lớn. CER chỉ giảm từ 7.27% xuống 7.06%, còn NED gần như tương đương giữa hai mô hình. Điều này cho thấy Attention Model có cải thiện, nhưng chưa thể xem là vượt trội tuyệt đối trong mọi trường hợp. CTC Baseline vẫn là một mô hình đơn giản, ổn định và cho kết quả tốt trong bài toán word-level HTR.

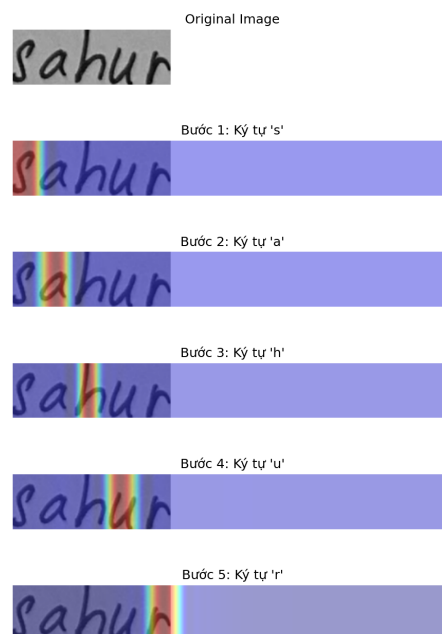
Một điểm quan trọng trong Attention Model là BiLSTM Context Encoder. Thành phần này giúp mỗi vị trí đặc trưng chứa thêm thông tin từ cả hai phía của từ viết tay, từ đó hỗ trợ attention tính trọng số tốt hơn khi giải mã từng ký tự. Đây cũng là một lý do khiến Attention Model đạt kết quả nhanh hơn baseline trong thực nghiệm.

Ngoài các chỉ số định lượng, việc quan sát trực tiếp một số mẫu dự đoán cũng giúp hiểu rõ hơn hành vi của mô hình. Bảng 6.5 minh họa một số trường hợp đúng và sai thường gặp.

Bảng 6.5: Một số ví dụ dự đoán của hai mô hình

Ảnh	Nhãn đúng	CTC dự đoán	Attention dự đoán	Nhận xét
Hình 1	example	example	example	Cả hai mô hình dự đoán đúng
Hình 2	writing	writng	writing	Attention dự đoán đầy đủ hơn
Hình 3	certain	cetrain	certain	CTC nhầm thứ tự một số ký tự
Hình 4	different	differnt	differnt	Cả hai mô hình còn thiếu ký tự

Một ưu điểm khác của Attention Model là có thể trực quan hóa attention map tại từng bước giải mã. Khi chạy dự đoán với tùy chọn `-save_attention`, hệ thống lưu attention map vào thư mục `outputs/attention_maps/`. Các bản đồ này giúp quan sát vùng ảnh mà mô hình tập trung khi dự đoán từng ký tự.



Hình 6.2: Ví dụ attention map mô hình Attention

Dù vậy, attention map chỉ nên được xem là công cụ hỗ trợ giải thích. Trong một số trường hợp, attention có thể tập trung lệch sang vùng trắng, vùng padding hoặc vùng nhiễu, đặc biệt khi ảnh đầu vào khác nhiều so với IAM Dataset.

6.4. Kiểm thử thực tế và phân tích lỗi

Bên cạnh đánh giá trên IAM Test Set, nhóm cũng thử nghiệm mô hình trên một số ảnh chữ viết tay thực tế. Mục đích của bước này là quan sát khả năng hoạt động của mô hình ngoài dữ liệu chuẩn.

Kết quả cho thấy mô hình hoạt động tốt hơn với các ảnh có đặc điểm gần với IAM, như chữ viết rõ, nền sạch, ảnh được crop sát vùng chữ và độ nghiêng không quá lớn. Ngược lại, với ảnh chụp bằng điện thoại, ánh sáng kém, nền không đồng đều hoặc nét chữ khác nhiều so với IAM, kết quả có thể giảm đáng kể.

Hiện tượng này phản ánh vấn đề domain shift. Vì vậy, kết quả tốt trên IAM Test Set chưa đủ để khẳng định mô hình có thể hoạt động tốt trong mọi bối cảnh thực tế.

Qua quan sát các trường hợp sai, lỗi của mô hình có thể chia thành một số nhóm chính:

- **Nhầm ký tự có hình dạng gần giống nhau:** ví dụ o và a, r và v, l và i.
- **Thiếu hoặc thừa ký tự:** thường xuất hiện ở các từ dài hoặc chữ viết liền nét.
- **Sai ở vùng chữ không rõ:** nét mờ, dính, nghiêng mạnh hoặc bị nhiễu.
- **Ảnh ngoài phân phối IAM:** nền, ánh sáng, độ dày nét và kiểu chữ khác biệt làm giảm chất lượng dự đoán.
- **Attention lệch vùng:** attention đôi khi tập trung vào padding hoặc vùng trắng thay vì nét chữ.

Các lỗi này cho thấy chất lượng dữ liệu, tiền xử lý và độ đa dạng của tập huấn luyện vẫn ảnh hưởng lớn đến hiệu quả nhận dạng.

6.5. Tổng kết chương

Kết quả thực nghiệm cho thấy cả hai mô hình đều học được đặc trưng chữ viết tay trên IAM Dataset và đạt kết quả tương đối tốt trên tập kiểm thử. Attention Model có Word Accuracy cao hơn CTC Baseline và CER thấp hơn một chút, nhưng mức cải thiện không quá lớn, đặc biệt khi xét theo NED.

CTC Baseline có ưu điểm là đơn giản, ổn định và hiệu quả. Attention Model linh hoạt hơn, có thể trực quan hóa attention map, nhưng phức tạp và phụ thuộc nhiều vào chất lượng căn chỉnh. Nhìn chung, kết quả trên IAM Test Set là tích cực, nhưng chưa phản ánh đầy đủ khả năng hoạt động trên ảnh chữ viết tay thực tế.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Trong phạm vi bài tập lớn, nhóm đã xây dựng một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ sử dụng PyTorch. Hệ thống nhận đầu vào là ảnh chứa một từ tiếng Anh viết tay và sinh ra chuỗi ký tự tương ứng. Dự án được triển khai hoàn toàn trên môi trường local, không sử dụng các dịch vụ OCR API bên thứ ba.

Nhóm đã thực hiện đầy đủ các bước chính của một pipeline học sâu cho bài toán nhận dạng chữ viết tay, bao gồm chuẩn bị dữ liệu, tiền xử lý ảnh, xây dựng vocabulary, thiết kế mô hình, huấn luyện, đánh giá và dự đoán ảnh thực tế. Bộ dữ liệu sử dụng trong dự án là IAM Handwriting Dataset ở mức word-level sau khi đã được làm sạch và chia thành các tập train, validation và test.

Về mặt mô hình, nhóm đã triển khai và so sánh hai kiến trúc. Mô hình thứ nhất là CTC Baseline, kết hợp ResNet18, BiLSTM và CTC Loss. Mô hình thứ hai là Attention Model, kết hợp ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.

Kết quả thực nghiệm trên tập kiểm thử IAM cho thấy CTC Baseline đạt 80.75% Word Accuracy và 7.27% CER. Attention Model đạt 83.09% Word Accuracy và 7.06% CER. Trên thiết lập thực nghiệm này, Attention Model cho kết quả tốt hơn CTC Baseline, đặc biệt ở chỉ số Word Accuracy. Tuy nhiên, mức cải thiện không quá lớn và cần được nhìn nhận trong phạm vi bộ dữ liệu cũng như điều kiện thực nghiệm hiện tại.

Dự án giúp nhóm hiểu rõ hơn về sự khác biệt giữa hai hướng tiếp cận phổ biến trong bài toán nhận dạng chuỗi từ ảnh. CTC có ưu điểm là đơn giản, ổn định và không cần decoder tự hồi quy. Trong khi đó, Attention Model linh hoạt hơn, có khả năng học căn chỉnh mềm giữa vùng ảnh và ký tự đầu ra, đồng thời hỗ trợ trực quan hóa attention map. Mỗi hướng tiếp cận đều có ưu điểm và hạn chế riêng.

7.2. Những kết quả đạt được

Trong quá trình thực hiện, nhóm đã đạt được một số kết quả chính sau:

- Xây dựng được pipeline chuẩn bị dữ liệu cho IAM Handwriting Dataset ở mức word-level.
- Cài đặt được mô hình CTC Baseline sử dụng ResNet18, BiLSTM và CTC Loss.
- Cài đặt được mô hình Attention sử dụng ResNet18, BiLSTM Context Encoder, Bahdanau Attention và GRU Decoder.
- Xây dựng được các script phục vụ huấn luyện, đánh giá và dự đoán ảnh đơn hoặc thư mục ảnh.
- Tích hợp các phương pháp giải mã gồm CTC Greedy Decoding, Attention Greedy Decoding và Beam Search.
- Đánh giá hai mô hình bằng các chỉ số Word Accuracy, CER và NED.

- Trực quan hóa được attention map đối với mô hình Attention để hỗ trợ phân tích kết quả.
- Tổ chức mã nguồn theo cấu trúc tương đối rõ ràng, tách riêng dữ liệu, mô hình, inference, utils và pipeline của từng mô hình.

Các kết quả trên cho thấy hệ thống đã đáp ứng được các mục tiêu cơ bản đặt ra ban đầu của bài tập lớn. Tuy vậy, hệ thống vẫn chủ yếu dừng ở mức nghiên cứu và thực nghiệm, chưa phải một sản phẩm OCR hoàn chỉnh có thể triển khai ổn định trong môi trường thực tế.

7.3. Hạn chế của dự án

Mặc dù hệ thống đạt được kết quả tương đối khả quan trên IAM Test Set, dự án vẫn còn một số hạn chế.

Thứ nhất, hệ thống mới xử lý bài toán nhận dạng ở mức từ đơn lẻ. Mỗi ảnh đầu vào được giả định chỉ chứa một từ tiếng Anh. Dự án chưa xử lý các bài toán phức tạp hơn như nhận dạng dòng văn bản, đoạn văn hoặc toàn bộ tài liệu. Các bước như phát hiện vùng chữ, tách dòng, tách từ và xử lý bố cục tài liệu chưa nằm trong phạm vi triển khai.

Thứ hai, mô hình phụ thuộc nhiều vào phân phối của IAM Dataset. Ảnh trong IAM thường đã được chuẩn hóa và cắt tương đối sát vùng chữ. Khi chuyển sang ảnh chữ viết tay thực tế, đặc biệt là ảnh chụp bằng điện thoại, nền không đồng đều, ánh sáng kém hoặc nét chữ khác nhiều, kết quả dự đoán có thể giảm đáng kể.

Thứ ba, khả năng tổng quát hóa của mô hình vẫn còn hạn chế. Kết quả tốt trên tập test của IAM không đồng nghĩa với việc mô hình có thể nhận dạng tốt mọi kiểu chữ viết tay ngoài thực tế. Đây là vấn đề thường gặp trong các mô hình học sâu khi dữ liệu triển khai khác với dữ liệu huấn luyện.

Thứ tư, mô hình Attention vẫn có thể gặp hiện tượng attention lệch sang vùng padding hoặc vùng trắng ở cuối ảnh. Nguyên nhân có thể đến từ việc chưa áp dụng attention padding mask đầy đủ trong quá trình tính trọng số attention.

Thứ năm, Beam Search trong dự án mới dừng ở mức giải mã dựa trên xác suất của mô hình, chưa kết hợp với Language Model ngoài. Vì vậy, mô hình chưa tận dụng được tri thức ngôn ngữ ở mức từ vựng hoặc ngữ cảnh rộng hơn.

Thứ sáu, dự án chưa tập trung tối ưu tốc độ suy luận, dung lượng mô hình hoặc khả năng đóng gói thành ứng dụng có giao diện người dùng. Các script hiện tại phù hợp cho thực nghiệm và kiểm thử hơn là triển khai thực tế.

7.4. Bài học rút ra

Thông qua quá trình thực hiện dự án, nhóm rút ra một số bài học.

Trước hết, đối với các bài toán học sâu, chất lượng dữ liệu và tiền xử lý ảnh có ảnh hưởng rất lớn đến kết quả. Ngay cả khi mô hình có kiến trúc tương đối tốt, ảnh đầu vào không phù hợp hoặc khác phân phối huấn luyện vẫn có thể làm kết quả suy giảm.

Bên cạnh đó, việc lựa chọn chỉ số đánh giá cũng rất quan trọng. Word Accuracy phản ánh khả năng dự đoán đúng hoàn toàn cả từ, trong khi CER cho cái nhìn chi tiết hơn ở cấp độ ký tự. Nếu chỉ nhìn vào một chỉ số, việc đánh giá mô hình có thể chưa đầy đủ.

Ngoài ra, việc so sánh baseline với mô hình cải tiến là cần thiết. CTC Baseline tuy đơn giản hơn nhưng vẫn đạt kết quả tốt, giúp nhóm có mốc tham chiếu rõ ràng khi đánh giá mô hình Attention. Điều này tránh việc chỉ nhìn vào một mô hình đơn lẻ mà không biết kết quả đó tốt hay chưa trong cùng điều kiện thực nghiệm.

Cuối cùng, kết quả thực nghiệm cần được diễn giải thận trọng. Mô hình có thể đạt kết quả tốt trên bộ dữ liệu chuẩn nhưng vẫn gặp khó khăn khi đưa ra ngoài thực tế. Do đó, khi đánh giá một hệ thống nhận dạng chữ viết tay, cần xét cả kết quả định lượng trên test set và kết quả kiểm thử bổ sung trên ảnh thực tế.

7.5. Hướng phát triển

Nếu tiếp tục phát triển, dự án có thể được cải thiện theo một số hướng sau.

7.5.1. Cải thiện tiền xử lý ảnh thực tế

Một hướng phát triển quan trọng là cải thiện pipeline tiền xử lý cho ảnh ngoài thực tế. Ảnh chụp bằng điện thoại thường có nền phức tạp, ánh sáng không đều, góc chụp lệch hoặc vùng chữ không được crop sát. Việc bổ sung các bước như làm sạch nền, cân bằng sáng, deskew, crop tự động và chuẩn hóa độ dày nét có thể giúp mô hình hoạt động ổn định hơn trên ảnh thực tế.

7.5.2. Bổ sung dữ liệu đa dạng hơn

Để cải thiện khả năng tổng quát hóa, có thể bổ sung thêm dữ liệu chữ viết tay từ nhiều người viết khác nhau, nhiều điều kiện chụp khác nhau và nhiều kiểu nét chữ khác nhau. Việc fine-tune mô hình trên dữ liệu thực tế đa dạng có thể giúp giảm khoảng cách giữa IAM Dataset và môi trường sử dụng thật.

7.5.3. Áp dụng Attention Padding Mask

Đối với mô hình Attention, có thể bổ sung attention padding mask để hạn chế mô hình tập trung vào vùng padding trắng. Điều này có thể giúp attention map ổn định hơn, đặc biệt với các ảnh có từ ngắn và nhiều khoảng trắng ở phía sau.

7.5.4. Kết hợp Language Model

Một hướng cải thiện khác là kết hợp mô hình nhận dạng với Language Model. Language Model có thể hỗ trợ lựa chọn chuỗi đầu ra hợp lý hơn, đặc biệt trong các trường hợp mô hình thị giác phân vân giữa nhiều ký tự có hình dạng gần giống nhau. Tuy nhiên, hướng này cần được thiết kế cẩn thận để tránh làm sai lệch kết quả đối với các từ ít gặp hoặc tên riêng.

7.5.5. Thử nghiệm các kiến trúc mới

Trong tương lai, có thể thử nghiệm các kiến trúc mạnh hơn như Transformer-based HTR, CRNN cải tiến hoặc mô hình kết hợp CTC–Attention. Các kiến trúc này có thể giúp mô hình học quan hệ dài hạn tốt hơn hoặc cải thiện quá trình căn chỉnh giữa ảnh và chuỗi ký tự.

7.5.6. Mở rộng sang line-level recognition

Dự án hiện chỉ xử lý word-level recognition. Một hướng phát triển tự nhiên là mở rộng sang nhận dạng dòng văn bản. Tuy nhiên, line-level HTR khó hơn vì chuỗi đầu ra dài hơn, ảnh có nhiều từ hơn và yêu cầu mô hình xử lý khoảng trắng cũng như cấu trúc dòng tốt hơn.

7.5.7. Xây dựng ứng dụng demo

Nếu có thêm thời gian, hệ thống có thể được đóng gói thành một ứng dụng demo đơn giản. Người dùng có thể tải ảnh lên, chọn mô hình CTC hoặc Attention, xem kết quả dự đoán và quan sát attention map. Ứng dụng demo sẽ giúp việc trình bày và kiểm thử hệ thống trực quan hơn.

7.6. Kết luận chung

Nhìn chung, dự án đã hoàn thành được mục tiêu chính là xây dựng và so sánh hai mô hình nhận dạng chữ viết tay mức độ từ đơn lẻ trên bộ dữ liệu IAM. Kết quả cho thấy mô hình Attention có cải thiện so với CTC Baseline trong thiết lập thực nghiệm của nhóm, nhưng sự cải thiện này cần được nhìn nhận trong phạm vi dữ liệu và điều kiện đánh giá hiện tại.

Hệ thống hiện tại phù hợp với mục tiêu học tập, thực nghiệm và so sánh mô hình trong khuôn khổ bài tập lớn. Tuy nhiên, để có thể sử dụng ổn định trong thực tế, mô hình cần được cải thiện thêm về dữ liệu, tiền xử lý, khả năng tổng quát hóa và cách triển khai. Đây cũng là những hướng mà nhóm có thể tiếp tục nghiên cứu nếu phát triển dự án ở các giai đoạn sau.

TÀI LIỆU THAM KHẢO

- [1] U. V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 369–376, 2006.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *International Conference on Learning Representations*, 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [5] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp. 1724–1734, 2014.
- [7] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [8] PyTorch Contributors, “PyTorch Documentation,” Available: <https://pytorch.org/docs/stable/index.html>.
- [9] PyTorch Contributors, “Torchvision Models and Pre-trained Weights,” Available: <https://pytorch.org/vision/stable/models.html>.
- [10] Albumentations Team, “Albumentations Documentation,” Available: <https://albumentations.ai/docs/>.
- [11] OpenCV Team, “OpenCV Documentation,” Available: <https://docs.opencv.org/>.
- [12] IAM Handwriting Database, “IAM Handwriting Database Official Website,” Available: <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>.